

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Hoàng Đức Minh

TÌM HIỂU KỸ THUẬT SINH TỰ ĐỘNG DỮ LIỆU
KIỂM THỬ TỪ ĐẶC TẢ CA SỬ DỤNG VÀ PHÁT
TRIỂN CÔNG CỤ HỖ TRỢ THEO HƯỚNG MÔ
HÌNH

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công Nghệ Thông Tin

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Hoàng Đức Minh

**TÌM HIỂU KỸ THUẬT SINH TỰ ĐỘNG DỮ LIỆU
KIỂM THỬ TỪ ĐẶC TẢ CA SỬ DỤNG VÀ PHÁT
TRIỂN CÔNG CỤ HỖ TRỢ THEO HƯỚNG MÔ
HÌNH**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công Nghệ Thông Tin

Cán bộ hướng dẫn: TS. Đặng Đức Hạnh

TÓM TẮT

Tóm tắt: Ngày nay, khi quá trình xây dựng đặc tả ca sử dụng là một trong những bước quan trọng nhất của việc phát triển phần mềm thì việc thẩm định và xác minh đặc tả ca sử dụng có vai trò quan trọng không nhỏ. Sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng có khả năng đánh giá tính đúng đắn hành vi của hệ thống được mô tả trong đặc tả ca sử dụng, đồng thời có thể dùng làm dữ liệu kiểm thử trong quá trình xây dựng kiểm thử chấp nhận. Vì vậy, khóa luận tập trung tìm hiểu một phương pháp sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng bằng các công cụ hỗ trợ theo hướng mô hình, trong đó đặc tả ca sử dụng được mô tả bằng ngôn ngữ FRSL sẽ được sử dụng để sinh tự động dữ liệu kiểm thử. Để làm được việc đó, khóa luận sử dụng công nghệ filmstrip để thẩm định hành vi của mô hình và công cụ tìm kiếm mô hình model validator được tích hợp trong công cụ USE. Tuy nhiên đặc tả ca sử dụng được mô tả bằng ngôn ngữ FRSL không thể dùng làm đầu vào trực tiếp cho công nghệ filmstrip, vậy nên khóa luận đề xuất một mô hình Filmstrip4Frsl được sử dụng làm trung gian cho các quá trình chuyển đổi từ mô hình FRSL đến mô hình filmstrip. Đồng thời, khóa luận sử dụng công nghệ Acceleo trên nền tảng Eclipse để chuyển đổi mô hình FRSL thành mô hình Filmstrip4Frsl. Dựa vào các công nghệ trên, ta có thể sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng được mô tả trong ngôn ngữ đặc tả yêu cầu FRSL.

Từ khóa: *Model2Text, Filmstrip, ngôn ngữ đặc tả yêu cầu (FRSL), sinh tự động dữ liệu kiểm thử.*

LỜI CẢM ƠN

Được sự phân công của Khoa Công nghệ thông tin, Trường Đại học Công Nghệ - ĐHQGHN và sự đồng ý của thầy giáo hướng dẫn – TS. Đặng Đức Hạnh, em đã hoàn thành đề tài: ***“Tìm hiểu kỹ thuật sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng và phát triển công cụ hỗ trợ theo hướng mô hình”***.

Để hoàn thành khóa luận này, em xin chân thành cảm ơn Thầy giáo hướng dẫn – TS. Đặng Đức Hạnh đã dìu dắt, tận tình hướng dẫn em trong suốt thời gian làm khóa luận.

Xin cảm ơn thầy cô Trường Đại học Công Nghệ đã giảng dạy tận tâm, nhiệt tình, chính điều đó đã giúp em có được những kiến thức nền tảng quý báu.

Xin cảm ơn các bạn trong cùng nhóm nghiên cứu đã hỗ trợ em nhiệt tình trong suốt quá trình làm khóa luận.

Sau cùng, em gửi lời biết ơn đến gia đình, người nuôi dưỡng, tạo điều kiện và động lực cho em học tập và yên tâm hoàn thành khóa luận này.

Dù đã cố gắng thực hiện đề tài một cách tốt nhất thế nhưng do ít được tiếp cận với tài liệu khoa học chuyên ngành phức tạp cùng với sự hạn chế về kiến thức và kinh nghiệm, khóa luận này của em không thể tránh khỏi những thiếu sót mà bản thân em chưa thấy được. Rất mong nhận được những lời nhận xét, sự góp ý của các thầy cô cùng các bạn sinh viên để bài khóa luận của em được hoàn chỉnh hơn.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Em – Hoàng Đức Minh, sinh viên QH-2018-I/CQ-CD Trường Đại học Công Nghệ xin cam đoan khóa luận tốt nghiệp ***“Tìm hiểu kỹ thuật sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng và phát triển công cụ hỗ trợ theo hướng mô hình”*** là công trình em tự thực hiện dưới sự đầu tư nghiên cứu nghiêm túc của bản thân và sự hướng dẫn của TS. Đặng Đức Hạnh. Trong khóa luận này em có sử dụng lại một số đoạn mã nguồn mở cùng mô hình được thầy cung cấp và giúp đỡ chỉnh sửa lại cho phù hợp với đề tài mà mình nghiên cứu, sẽ không có phần nào được sao chép từ tài liệu bên ngoài mà không được dẫn nguồn. Em xin chịu mọi trách nhiệm về lời cam đoan này.

Hà Nội, ngày tháng năm 2022

Sinh viên

Hoàng Đức Minh

MỤC LỤC

TÓM TẮT.....	3
LỜI CẢM ƠN.....	4
LỜI CAM ĐOAN.....	5
MỤC LỤC.....	6
DANH MỤC TỪ VIẾT TẮT.....	9
DANH MỤC HÌNH VẼ.....	10
DANH MỤC BẢNG.....	13
MỞ ĐẦU.....	1
CHƯƠNG 1. KIẾN THỨC NỀN TẢNG.....	4
1.1. Giới thiệu.....	4
1.2. Đặc tả yêu cầu chức năng.....	4
1.3. Kỹ thuật tìm kiếm mô hình với ModelValidator.....	7
1.3.1. Ngôn ngữ mô hình hóa thống nhất (UML).....	7
1.3.2. Ngôn ngữ ràng buộc đối tượng (OCL).....	8
1.3.3. Giới thiệu về công cụ USE.....	9
1.3.4. Model Validator.....	10
1.4. Biểu diễn mô hình kịch bản với Filmstrip.....	12
1.4.1. Ý tưởng.....	13
1.4.2. Chuyển đổi từ mô hình ứng dụng sang mô hình filmstrip.....	14
1.4.3 Phương pháp tạo ràng buộc khung PCDL.....	18
1.4.3.1. Phân biệt các hành vi thêm mới, loại bỏ và giữ nguyên đối tượng...19	
1.4.3.2. Biểu diễn hậu điều kiện của phương thức dưới dạng bảng PCDL....20	
1.4.3.3. Chuyển đổi từ mô hình PCDL sang hậu điều kiện.....21	
1.6. Một số công nghệ chuyển đổi mô hình.....	22
1.7. Tổng kết chương.....	24
CHƯƠNG 2. PHƯƠNG PHÁP SINH TỰ ĐỘNG DỮ LIỆU KIỂM THỬ TỪ ĐẶC TẢ CA SỬ DỤNG FRSL.....	25
2.1. Giới thiệu.....	25

2.2. Tổng quan về phương pháp.....	25
2.3. Biểu diễn mô hình Filmstrip4Frsl.....	27
2.3.1. Yêu cầu của Filmstrip4Frsl.....	27
2.3.2. Biểu diễn Filmstrip4Frsl dưới dạng mô hình ứng dụng.....	28
2.3.2.1. Các thành phần chính của Filmstrip4Frsl ở dạng mô hình ứng dụng	28
2.3.2.2. Bất biến lớp và tiên, hậu điều kiện cho các phương thức.....	31
2.3.3. Biểu diễn Filmstrip4Frsl dưới dạng mô hình filmstrip.....	37
2.4. Chuyển đổi từ mô hình FRSL sang mô hình Filmstrip4Frsl.....	39
2.4.1. Luật chuyển đổi mô hình từ FRSL sang mô hình Filmstrip4Frsl.....	39
2.4.2. Luật chuyển đổi tệp cấu hình cho Model Validator.....	42
2.5. Sinh dữ liệu ca kiểm thử từ mô hình Filmstrip4Frsl.....	44
2.6. Tổng kết chương.....	47
CHƯƠNG 3. CÀI ĐẶT VÀ THỰC NGHIỆM.....	48
3.1. Giới thiệu.....	48
3.2. Công cụ hỗ trợ.....	48
3.3. Bài toán thực nghiệm.....	51
3.4. Đánh giá.....	55
KẾT LUẬN.....	57
DANH MỤC TÀI LIỆU THAM KHẢO.....	58

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Nguyên nghĩa	Nghĩa tiếng Việt
FRSL	Functional Requirements Specification Language	Ngôn ngữ đặc tả yêu cầu chức năng
USE	UML-based Specification Enviroment	Môi trường đặc tả dựa trên UML
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa thống nhất
OCL	Object Constraint Language	Ngôn ngữ ràng buộc đối tượng
PCDL	Post-/frame condition determining language	Ngôn ngữ xác định hậu điều kiện /ràng buộc khung
OMG	Object Management Group	Nhóm quản lý đối tượng

DANH MỤC HÌNH VẼ

Hình 1.1 Bảng mẫu đặc tả ca sử dụng.

Hình 1.2 Siêu mô hình FRSL dưới dạng đồ họa.

Hình 1.3 Cú pháp trừu tượng dưới dạng cây.

Hình 1.4 Cú pháp cụ thể của ngôn ngữ FRSL.

Hình 1.5 Các loại biểu đồ UML.

Hình 1.6 Mô hình lớp Person.

Hình 1.7 Một bất biến lớp của lớp Person.

Hình 1.8 Tiên và hậu điều kiện của một phương thức.

Hình 1.9 Giao diện công cụ USE [13].

Hình 1.10 Biểu đồ đối tượng và biểu đồ lớp của mô hình Person.

Hình 1.11 Tập cấu hình sinh dữ liệu và giao diện của plugin Model Validator.

Hình 1.12 Biểu đồ lớp của mô hình ứng dụng trên công cụ USE.

Hình 1.13 Biểu đồ lớp của mô hình filmstrip.

Hình 1.14 Mối tương quan giữa biểu đồ tuần tự của mô hình ứng dụng và biểu đồ đối tượng của mô hình filmstrip.

Hình 1.15 Các luật chuyển đổi từ mô hình ứng dụng sang mô hình filmstrip [11].

Hình 1.16 Kết quả sau khi thực hiện tìm kiếm mô hình bằng Model Validator.

Hình 1.17 Miền của các đối tượng được thêm mới, loại bỏ và giữ nguyên trong quá trình thực hiện phương thức [6].

Hình 1.18 Ví dụ tổng quát. Biểu đồ lớp (bên trái) và biểu diễn dạng bảng của mô hình PCDL (bên phải) [6].

Hình 1.19 Phương thức tổng quát chuyển đổi từ giá trị trong bảng sang hậu điều kiện OCL [6].

Hình 1.20 Các bước sinh mã nguồn của Acceleo.

Hình 2.1 Toàn bộ quy trình thực hiện phương pháp sinh tự động dữ liệu từ đặc tả ca sử dụng.

Hình 2.2 Ví dụ. Ca sử dụng BuyTicket biểu diễn bằng ngôn ngữ FRSL.

Hình 2.3 Biểu diễn lớp tham gia ca sử dụng cùng các quan hệ của mô hình Filmstrip4Frsl bằng cú pháp của công cụ USE.

Hình 2.4 Ví dụ về một quan hệ không hợp lệ trong Filmstrip4Frsl.

Hình 2.5 Lớp ca sử dụng và các quan hệ trong Filmstrip4Frsl (bên trái) và lớp ca sử dụng trong FRSL (bên phải).

Hình 2.6 Toàn bộ các lớp và quan hệ trong mô hình Filmstrip4Frsl ở dạng mô hình ứng dụng của ca sử dụng BuyTicket.

Hình 2.7 Bất biến lớp ràng buộc lớp BuyTicket chỉ có duy nhất 1 đối tượng.

Hình 2.8 Bất biến lớp ràng buộc quan hệ giữa lớp ca sử dụng và lớp tham gia ca sử dụng.

Hình 2.9 Biểu đồ lớp của mô hình Filmstrip4Frsl ở dạng mô hình filmstrip khi chưa mô tả tiền và hậu điều kiện của các phương thức.

Hình 2.10 Ràng buộc thứ tự thực hiện của các phương thức.

Hình 2.11 Tiền và hậu điều kiện hoàn chỉnh của các phương thức.

Hình 2.12 Biểu diễn mô hình Filmstrip4Frsl dưới dạng mô hình filmstrip.

Hình 2.13 Biểu đồ đối tượng của mô hình Filmstrip4Frsl dưới dạng mô hình filmstrip sau khi đã được sinh dữ liệu.

Hình 2.14 Mô hình filmstrip được sinh ra dưới cú pháp của công cụ USE.

Hình 2.15 Truy vấn để lấy toàn bộ các phương thức thành một OrderedSet.

Hình 2.16 Khuôn mẫu để tạo bất biến lớp ràng buộc có một đối tượng lớp ca sử dụng.

Hình 2.17 Khuôn mẫu để tạo bất biến lớp ràng buộc quan hệ giữa lớp ca sử dụng và các lớp tham gia ca sử dụng.

Hình 2.18 Khuôn mẫu để sinh ràng buộc thứ tự thực hiện của phương thức.

Hình 2.19 Định dạng cấu hình của một đối tượng.

Hình 2.20 Sự chuyển đổi của mô hình Filmstrip4Frsl từ dạng mô hình ứng dụng sang dạng mô hình filmstrip.

Hình 2.21 Tập cấu hình sinh dữ liệu đầy đủ.

Hình 2.22 Biểu đồ đối tượng được sinh ra sau khi sử dụng plugin Model Validator.

Hình 3.1 Quy trình và các công cụ được sử dụng để sinh dữ liệu kiểm thử từ đặc tả ca sử dụng.

Hình 3.2 Ca sử dụng được mô tả theo cú pháp của ngôn ngữ FRSL (bên trái) và được biểu diễn theo cấu trúc cây (bên phải).

Hình 3.3 Mô hình Filmstrip4Frsl được sinh ra theo cú pháp của công cụ USE.

Hình 3.4 Giao diện của plugin filmstrip.

Hình 3.5 Giao diện của plugin Model Validator.

Hình 3.6 Ca sử dụng HandleCashPayment được mô tả bằng ngôn ngữ FRSL.

Hình 3.7 Biểu đồ lớp mô hình Filmstrip4Frsl cho ca sử dụng HandleCashPayment.

Hình 3.8 Tập cấu hình sinh dữ liệu cho ca sử dụng HandleCashPayment.

Hình 3.9 Các ràng buộc của mô hình Filmstrip4Frsl.

Hình 3.10 Mô hình Filmstrip4Frsl sau khi được chuyển đổi bằng plugin filmstrip.

Hình 3.11 Biểu đồ đối tượng được sinh ra thể hiện sự biến đổi trạng thái của các đối tượng trong ca sử dụng HandleCashPayment.

DANH MỤC BẢNG

Bảng 2.1 Mô hình dạng bảng của PCDL áp dụng cho Filmstrip4Frsl

MỞ ĐẦU

Ngày nay, các ứng dụng của công nghệ thông tin đã trở thành một phần không thể thiếu trong cuộc sống hằng ngày của mỗi người. Các cuộc cách mạng trong công nghệ đang được diễn ra trên toàn thế giới, số lượng nghiên cứu và các hoạt động trong lĩnh vực công nghệ thông tin ngày càng lớn, cung cấp các dịch vụ thiết yếu cho xã hội. Đặc biệt hơn, sự xuất hiện của vi rút covid 19 đã thúc đẩy mạnh mẽ tới việc phát triển công nghệ, đẩy nhanh quá trình chuyển đổi số. Phát triển ứng dụng phần mềm trở thành một trong những thành phần quan trọng nhất, là ưu tiên của quốc gia để bước vào cuộc đua công nghệ trên toàn thế giới. Tuy nhiên, các hệ thống phần mềm càng ngày càng trở nên phức tạp, các thách thức ngày một nhiều lên khiến việc phát triển phần mềm trở nên không hề dễ dàng.

Bên cạnh đó, với sự thay đổi và biến chuyển không ngừng trong công nghệ, một ứng dụng phần mềm có thể trở nên lỗi thời nếu không phát triển liên tục khi nhu cầu ngày một gia tăng. Với sự phức tạp của các ứng dụng, việc thay đổi, nâng cấp phần mềm là một vấn đề không hề đơn giản. Tự động hóa trong quá trình phát triển phần mềm trở thành một điều tất yếu, giúp giảm thiểu phần lớn các quy trình được thực hiện trong quá trình xây dựng hệ thống. Vì vậy, phương pháp phát triển phần mềm hướng mô hình (MDE) ra đời để giải quyết bài toán trên. Ý tưởng của phương pháp trên là mô hình hóa các đối tượng ở thế giới thực, nâng mức độ trừu tượng trong việc phát triển phần mềm lên một tầm cao mới. Trừu tượng hóa là cốt lõi của phát triển phần mềm từ những ngày đầu tiên, giúp giảm thiểu sự phức tạp, là yếu tố then chốt khiến phát triển phần mềm trở nên dễ dàng tiếp cận và phổ biến như hiện nay. Kỹ thuật phát triển hướng mô hình tập trung vào mô hình hóa các đối tượng, quá trình chuyển đổi giữa các mô hình. Việc sử dụng kỹ thuật hướng mô hình vào phát triển phần mềm có thể giúp việc xây dựng phần mềm trở nên dễ dàng hơn, chuyển đổi giữa các mô hình sẽ cung cấp cho nhà thiết kế nhiều góc nhìn về hệ thống được xây dựng, dẫn đến việc tăng năng suất cũng như chất lượng của sản phẩm phần mềm.

Ngôn ngữ mô hình hóa FRSL là một ngôn ngữ được xây dựng áp dụng theo kỹ thuật phát triển hướng mô hình. Ngôn ngữ này được sinh ra với vai trò mô tả ca sử dụng, biểu diễn ca sử dụng dưới dạng mô hình và xác định các hành vi trong ca sử dụng để từ đó xác định được các yêu cầu chức năng của hệ thống. Xây dựng ca sử dụng là một thành phần quan trọng, có thể được sử dụng làm đầu vào để xây dựng nhiều thành phần, tạo tác khác trong quá trình phát triển phần mềm. Mô hình hóa ca sử dụng

có thể giúp tự động hóa trong quá trình xây dựng các thành phần quan trọng khác, một ví dụ điển hình có thể kể đến như kiểm thử chấp nhận của hệ thống.

Tuy nhiên, việc thẩm định tính chính xác các hành vi trong ca sử dụng là một điều rất phức tạp, tốn nhiều thời gian và phụ thuộc vào người thiết kế. Chính vì vậy, khóa luận đề xuất một phương pháp sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng được mô tả bằng ngôn ngữ mô hình hóa FRSL. Điều này sẽ giúp người thiết kế kiểm tra các hành vi ca sử dụng ngay từ pha thiết kế ca sử dụng, giảm thiểu được nhiều rủi ro cũng như sai sót trong quá trình phát triển sau này. Đồng thời, dữ liệu được sinh ra có thể được sử dụng cho các ca kiểm thử chấp nhận của hệ thống, tiến đến xây dựng các ca kiểm thử chấp nhận từ ca sử dụng tự động hoàn toàn.

Khóa luận sẽ tìm hiểu và giới thiệu phương pháp sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng bằng các công cụ được phát triển theo hướng mô hình. Khóa luận sẽ mô tả các bước để thực hiện phương pháp sinh tự động dữ liệu. Tại đây, khóa luận sẽ mô tả tổng quan các thành phần và quá trình thực hiện, giúp người đọc có cái nhìn tổng quát về toàn bộ phương pháp.

Tiếp theo, khóa luận đề xuất mô hình Filmstrip4Frsl. Filmstrip4Frsl là một mô hình trung gian giúp chuyển đổi từ mô hình nguồn là FRSL đến mô hình đích là mô hình filmstrip. Các luật chuyển đổi chuyển đổi từ mô hình FRSL sang mô hình Filmstrip4Frsl bằng công cụ Accelele cũng sẽ được mô tả một cách rõ ràng.

Cuối cùng, khóa luận sẽ trình bày phương pháp sinh dữ liệu từ mô hình Filmstrip4Frsl bằng các plugin filmstrip và model validator của công cụ USE. Dữ liệu sinh ra sẽ được thể hiện dưới dạng biểu đồ đối tượng.

Cấu trúc khóa luận gồm 5 phần như sau:

Mở đầu: Giới thiệu chung về kỹ nghệ ngôn ngữ phần mềm trong bối cảnh hiện nay, đặt vấn đề và đưa ra định hướng nghiên cứu.

Chương 1. Kiến thức nền tảng: Đưa ra các kiến thức nền tảng về đặc tả ca sử dụng bằng ngôn ngữ FRSL các công nghệ sinh tự động dữ liệu, thẩm định mô hình filmstrip, công nghệ chuyển đổi mô hình Accelele.

Chương 2. Phương pháp sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng FRSL: Mô tả quá trình thực hiện sinh tự động dữ liệu từ đặc tả ca sử dụng, giới thiệu mô hình đề xuất Filmstrip4Frsl và các luật chuyển đổi

Chương 3. Cài đặt và thực nghiệm: Vận dụng, thực nghiệm và đánh giá kỹ thuật sinh dữ liệu kiểm thử từ đặc tả ca sử dụng bằng nền tảng Eclipse và công cụ USE.

Kết luận: Đưa ra tổng kết về đề tài, các đề xuất và hướng nghiên cứu tiếp theo.

CHƯƠNG 1. KIẾN THỨC NỀN TẢNG

Trong chương này, khóa luận sẽ giới thiệu những kiến thức nền tảng về kỹ nghệ hướng mô hình và đặc tả yêu cầu chức năng nhằm cung cấp cho người đọc cơ sở trước khi đi vào bài toán và phương pháp giải quyết.

1.1. Giới thiệu

Các kiến thức nền tảng về khái niệm, công nghệ cho kỹ thuật sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng sẽ được cung cấp trong Chương 1. Trong đó, Mục 1.2 nêu lên định nghĩa và các kiến thức về đặc tả chức năng cũng như giới thiệu và mô tả cách biểu diễn đặc tả yêu cầu bằng ngôn ngữ FRSL. Mục 1.3 trình bày về ngôn ngữ mô hình hóa thống nhất UML, ngôn ngữ ràng buộc đối tượng OCL, công cụ phát triển mô hình hóa USE và mô tả plugin tìm kiếm mô hình ModelValidator được tích hợp trong công cụ USE. Ở Mục 1.4, tìm hiểu và giới thiệu plugin Filmstrip được sử dụng để kiểm tra và thẩm định hành vi của mô cùng với phương pháp xây dựng ràng buộc khung PCDL. Mục 1.5 sẽ chỉ ra một số kiến thức về công nghệ chuyển đổi mô hình và giới thiệu về công nghệ chuyển đổi mô hình Acceleo được sử dụng trong khóa luận.

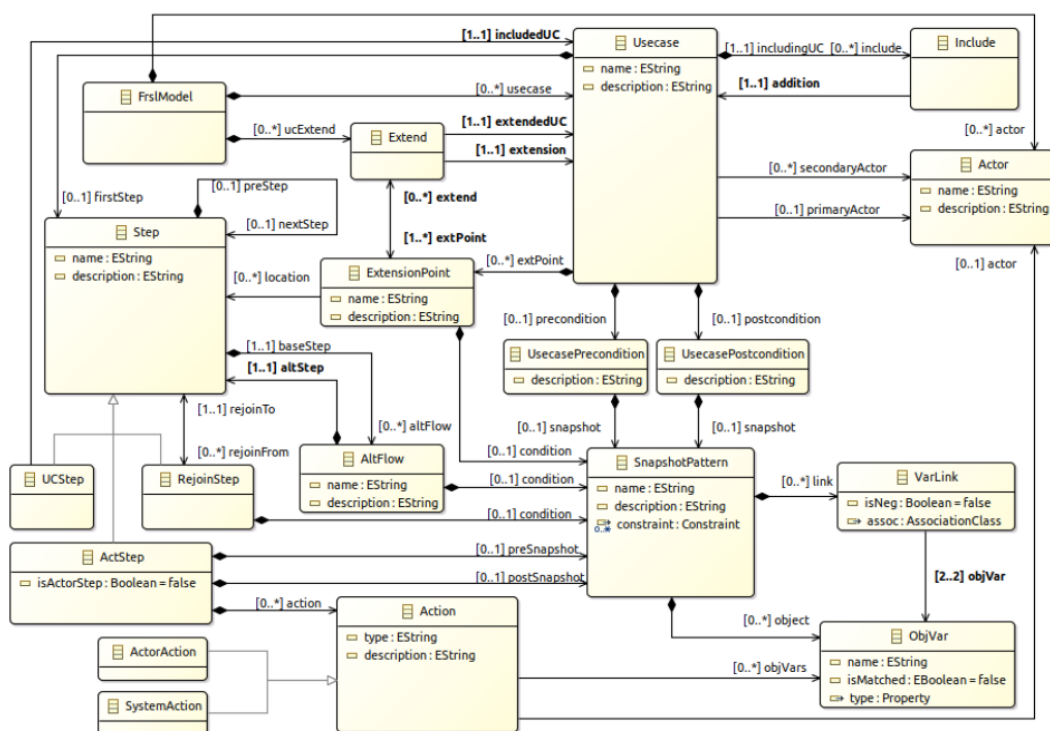
1.2. Đặc tả yêu cầu chức năng

Trong phần này, tổng quan về đặc tả chức năng cùng mô tả ngôn ngữ mô hình hóa FRSL sẽ được giới thiệu.

Ca sử dụng là một thành phần quan trọng trong quá trình phân tích hệ thống, làm rõ các chức năng, yêu cầu của hệ thống. Ca sử dụng thể hiện tương tác của người dùng với hệ thống trong các điều kiện và môi trường cụ thể, nhằm thực hiện một mục tiêu nào đó. Ca sử dụng mô tả hành vi của hệ thống khi có sự tác động của các tác nhân (actor), mục tiêu của sự tác động đó,... Để làm được việc đó, một ca sử dụng cần mô tả được các yếu tố sau: yêu cầu chức năng của ca sử dụng, các tác nhân tham gia ca sử dụng, mục đích/kết quả khi tham gia ca sử dụng và kịch bản thực hiện - bao gồm các sự kiện để dẫn đến mục tiêu. Ca sử dụng có thể được mô tả dưới dạng bảng mẫu đặc tả ca sử dụng như trong Hình 1.1.

Add Comment
Brief description: The actor adds a comment
Actors: Participant
Preconditions: The actor is a registered student
Basic flow of events: 1. The actor logs into the system 2. The system authenticates the actor and starts a session 3. The actor chooses to add a comment 4. The actor is guided by the system to fill in required information to add a comment 5. The system acknowledges that comment added 6. The actor leaves the system
Extensions: 1a. The system fails to authenticate the actor. • The system informs the actor and doesn't allow the actor to proceed 3a. The system fails to add a comment • The system informs the actor
Post-conditions: The new comment is added
Special requirements: Actor is connected to Internet and uses a Java enabled Mobile phone

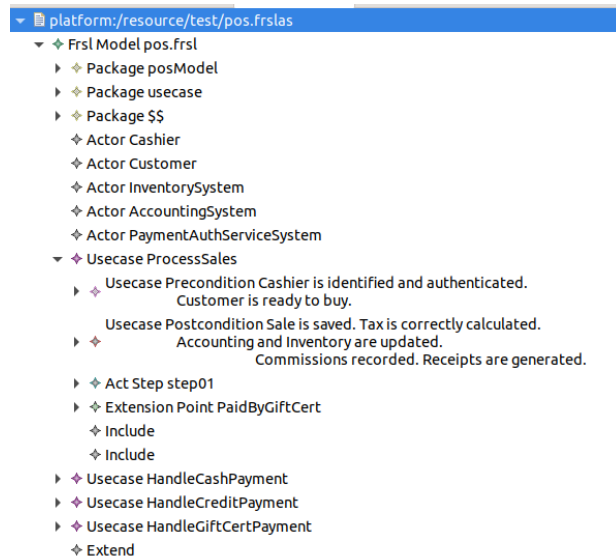
Hình 1.1 Bảng mẫu đặc tả ca sử dụng.



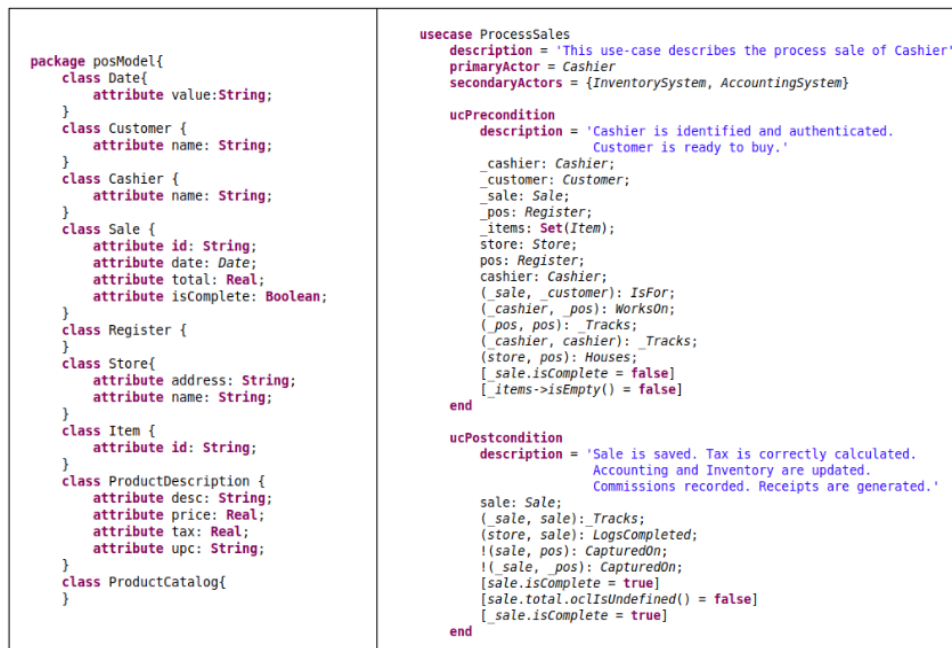
Hình 1.2 Siêu mô hình FRSL dưới dạng đồ họa.

Ngôn ngữ đặc tả yêu cầu FRSL là ngôn ngữ được xây dựng với mục đích mô hình hóa đặc tả yêu cầu chức năng, cung cấp cho người dùng các cú pháp để thiết kế ca sử dụng với đầy đủ thông tin. Cú pháp trừu tượng của ngôn ngữ FRSL tuân theo siêu mô hình, biểu diễn dưới dạng đồ họa như trong Hình 1.2. Cú pháp này được sử

dùng để mô tả ca sử dụng dưới dạng một tệp có định dạng XMI (XML Metadata Interchange) thể hiện dưới dạng cây trong Hình 1.3. Cú pháp cụ thể dạng văn bản của FRSL sẽ được định nghĩa từ các luật ngữ pháp của cú pháp trừu tượng, giúp người dùng mô tả dưới dạng văn bản thể hiện trong Hình 1.4. Từ đó, người dùng có thể chuyển đổi đặc tả yêu cầu ở dạng văn bản (tuân theo cú pháp cụ thể) sang dạng mô hình (tuân theo cú pháp trừu tượng), từ đó có thể sử dụng để lấy thông tin hoặc chuyển đổi sang các tạo tác khác.



Hình 1.3 Cú pháp trừu tượng dưới dạng cây.



Hình 1.4 Cú pháp cụ thể của ngôn ngữ FRSL.

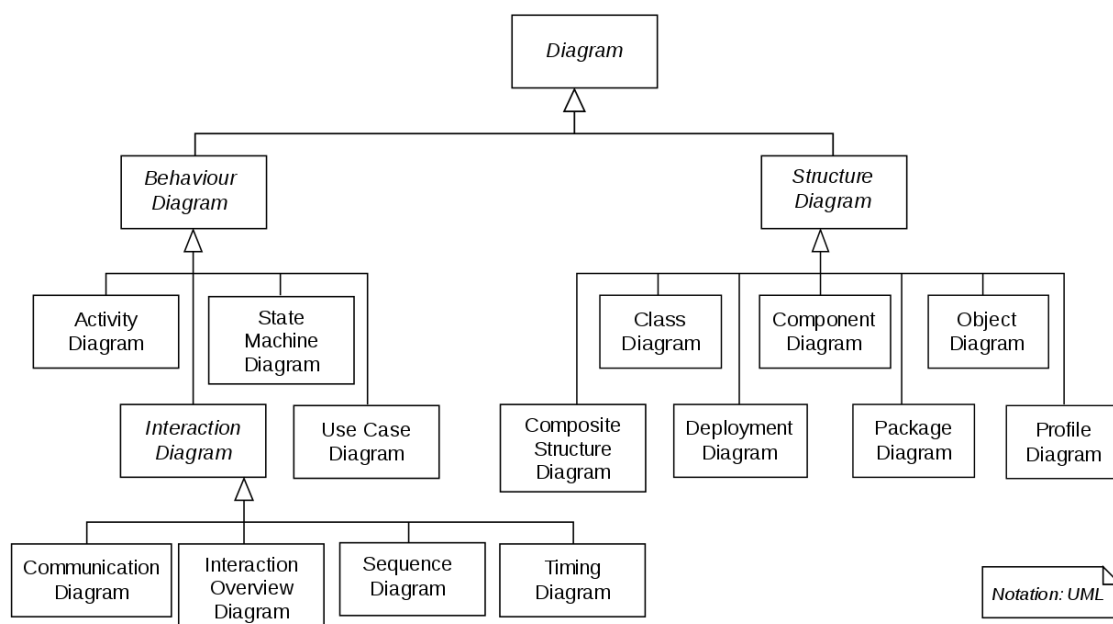
1.3. Kỹ thuật tìm kiếm mô hình với ModelValidator

Phần này sẽ cung cấp các kiến thức về ngôn ngữ mô hình hóa, ngôn ngữ ràng buộc đối tượng là những nền tảng của khóa luận. Ngoài ra, công cụ USE cũng như kỹ thuật tìm kiếm mô hình với plugin ModelValidator được sử dụng trong công cụ USE sẽ được giới thiệu trong phần này.

1.3.1. Ngôn ngữ mô hình hóa thống nhất (UML)

Ngôn ngữ mô hình thống nhất (UML) là một ngôn ngữ mô hình hóa được sử dụng để biểu diễn, xây dựng và đặc tả các khía cạnh của một hệ thống phần mềm dưới dạng hình ảnh [9]. Ngày nay, khi các hệ thống phần mềm càng ngày càng phát triển cả về kích thước, độ phức tạp khiến quá trình xây dựng và bảo trì hệ thống càng trở nên quan trọng. Chính vì vậy, việc sử dụng ngôn ngữ ở một mức trừu tượng cao hơn đó là ngôn ngữ mô hình hóa UML sẽ giúp giảm đi sự phức tạp cũng như công sức để xây dựng hệ thống, thay vào đó tập trung vào việc cung cấp chính xác các thông tin quan trọng về mọi khía cạnh của hệ thống để thiết kế và phát triển phần mềm.

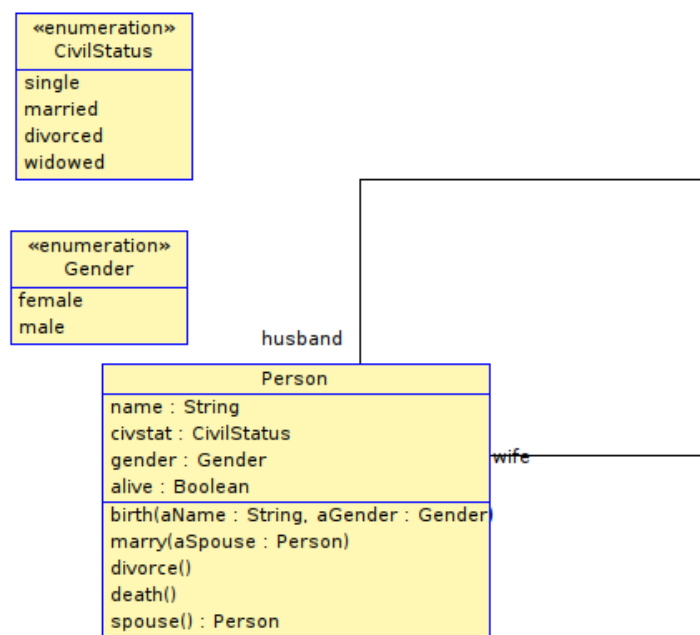
UML là một ngôn ngữ thống nhất, là một tập hợp các biểu đồ khác nhau để mô tả hệ thống từ nhiều khía cạnh khác nhau. Các biểu đồ này có thể chia thành hai nhóm, nhóm đầu là các biểu đồ cấu trúc - mô tả các thành phần tĩnh của một hệ thống, trong khi nhóm thứ hai là các biểu đồ hành vi - mô tả các thành phần động của hệ thống. Có tất cả 15 loại biểu đồ của UML được chia thành hai nhóm theo Hình 1.5.



Hình 1.5 Các loại biểu đồ UML.

1.3.2. Ngôn ngữ ràng buộc đối tượng (OCL)

UML là một ngôn ngữ thể hiện các khía cạnh của hệ thống dưới dạng hình ảnh, biểu đồ, chỉ có thể hỗ trợ một vài các ràng buộc cơ bản vì giới hạn của một ngôn ngữ mô hình hóa. Vậy nên chỉ sử dụng UML là không đủ để có thể xây dựng một mô hình chính xác và nhất quán nếu chỉ sử dụng UML. Đây là lúc ngôn ngữ ràng buộc đối tượng (OCL) thể hiện vai trò của mình. OCL là ngôn ngữ được xây dựng và phát triển bởi Object Management Group (OMG) vào năm 1997, với vai trò tập trung mô tả mô hình UML một cách chính xác nhau và từ đó có thể mở rộng được khả năng của UML. OCL là một ngôn ngữ có *kiểu xác định, khai báo* (declarative) và là ngôn ngữ *không có hiệu ứng phụ* (side effect-free specification language) [9]. OCL là ngôn ngữ có *kiểu xác định* có nghĩa mỗi biểu thức OCL đều có kiểu dữ liệu nhất định, có các luật cũng như các hàm được định nghĩa của kiểu đó. Ngôn ngữ *khai báo* thể hiện OCL không thể hiện cách thức thực hiện mà chỉ tập trung đến việc khai báo. Cuối cùng, *không có hiệu ứng phụ* có ý nghĩa các biểu thức OCL sẽ chỉ truy vấn hoặc thể hiện các ràng buộc mà không thay đổi trạng thái của đối tượng.



Hình 1.6 Mô hình lớp Person.

OCL có thể được sử dụng để truy vấn với các biểu thức OCL hoặc để xác định các giới hạn mà mô hình cần phải thỏa mãn với ràng buộc OCL. Ràng buộc OCL có thể được chia thành ba loại, bao gồm bất biến lớp, tiền và hậu điều kiện của phương thức (operation). Các ràng buộc OCL đều được biểu diễn dưới dạng các bất biến được

xác định trong một ngữ cảnh cụ thể, thể hiện với từ khóa *context*. Phần thân của ràng buộc là một điều kiện boolean, phải được thỏa mãn với mọi đối tượng được xác định trong ngữ cảnh đó. Với bất biến lớp, tất cả các đối tượng của một lớp phải thỏa mãn các điều kiện được xác định trong phần thân. Ràng buộc OCL bên dưới thể hiện một bất biến lớp của lớp *Person*, trong đó những đối tượng lớp *Person* có thuộc tính *gender* là *female* sẽ không có liên kết với đối tượng *Person* khác có vai trò *wife*.

```
context Person inv traditionalRoles:
    (gender=#female implies wife→isEmpty)
```

Hình 1.7 Một bất biến lớp của lớp Person.

Với tiền và hậu điều kiện của phương thức, tương tự như bất biến lớp, các ràng buộc được mô tả ở phần thân phải được thỏa mãn với tất cả đối tượng khi thực hiện phương thức. Tuy nhiên, tiền điều kiện chỉ được áp dụng trước khi thực hiện phương thức, còn hậu điều kiện sẽ được áp dụng sau khi thực hiện phương thức. Ở hậu điều kiện ta có thể truy cập trạng thái của một đối tượng với từ khóa *@pre*. Ví dụ về tiền và hậu điều kiện được biểu diễn bằng OCL sẽ được biểu diễn ngay dưới đây.

```
context Person inv traditionalRoles:
    (gender=#female implies wife→isEmpty)
context Person::marry(aSpouse:Person)
pre unmarried:
    self.spouse()→isEmpty and aSpouse.spouse()→isEmpty and
    Set{self.gender,aSpouse.gender}=Set{#female,#male}
post married:
    Set{aSpouse}=self.spouse() and Set{self}=aSpouse.spouse()
post: result=meetings→select(isConfirmed)→size()
```

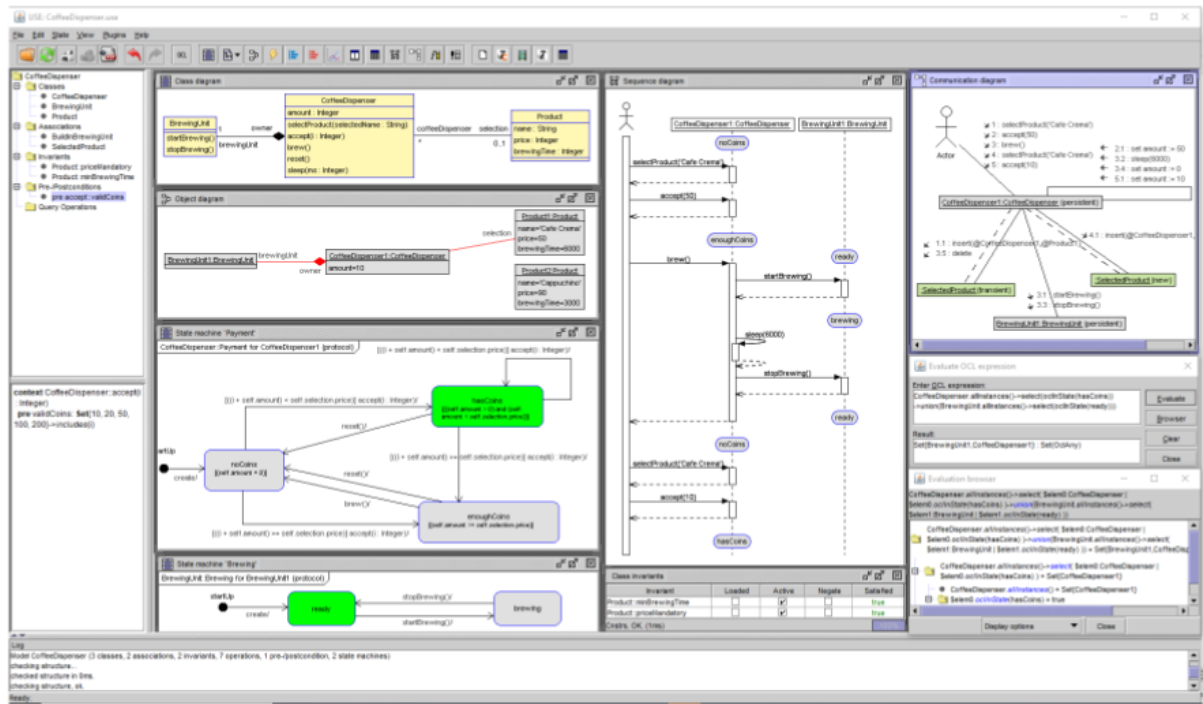
Hình 1.8 Tiền và hậu điều kiện của một phương thức.

1.3.3. Giới thiệu về công cụ USE

Công cụ USE (UML-based Specification Enviroment) là một công cụ được sử dụng để đặc tả, mô hình hóa và kiểm thử mô hình UML/OCL [12]. Công cụ này được phát triển bằng ngôn ngữ Java và cung cấp cho người dùng khả năng tích hợp plugin vào chính công cụ này. Trong USE, các mô hình UML/OCL được mô tả và biểu diễn dưới dạng văn bản lưu trong tệp có định dạng *.use*. Tệp này lưu các thông tin về các lớp, thuộc tính của các lớp, các phương thức, quan hệ giữa các lớp cùng các ràng buộc. Các ràng buộc này được thể hiện bằng ngôn ngữ OCL để thể hiện tiền và hậu điều kiện của phương thức hoặc các bất biến lớp. Công cụ USE sẽ lấy các thông tin trên rồi hiển thị mô hình dưới dạng giao diện, kèm theo đó là các biểu đồ lớp, biểu đồ đối tượng,

biểu đồ tuần tự cũng như biểu đồ trạng thái v.v. Ngoài ra, USE còn cung cấp cho người dùng khả năng cũng cấp dữ liệu cho các mô hình, truy vấn và thay đổi trạng thái các đối tượng trong mô hình bằng ngôn ngữ soil (simple OCL-like imperative programming language).

Hình 1.9 dưới đây mô tả các tính năng của công cụ USE bao gồm biểu diễn biểu đồ lớp, biểu đồ đối tượng, biểu đồ tuần tự,...



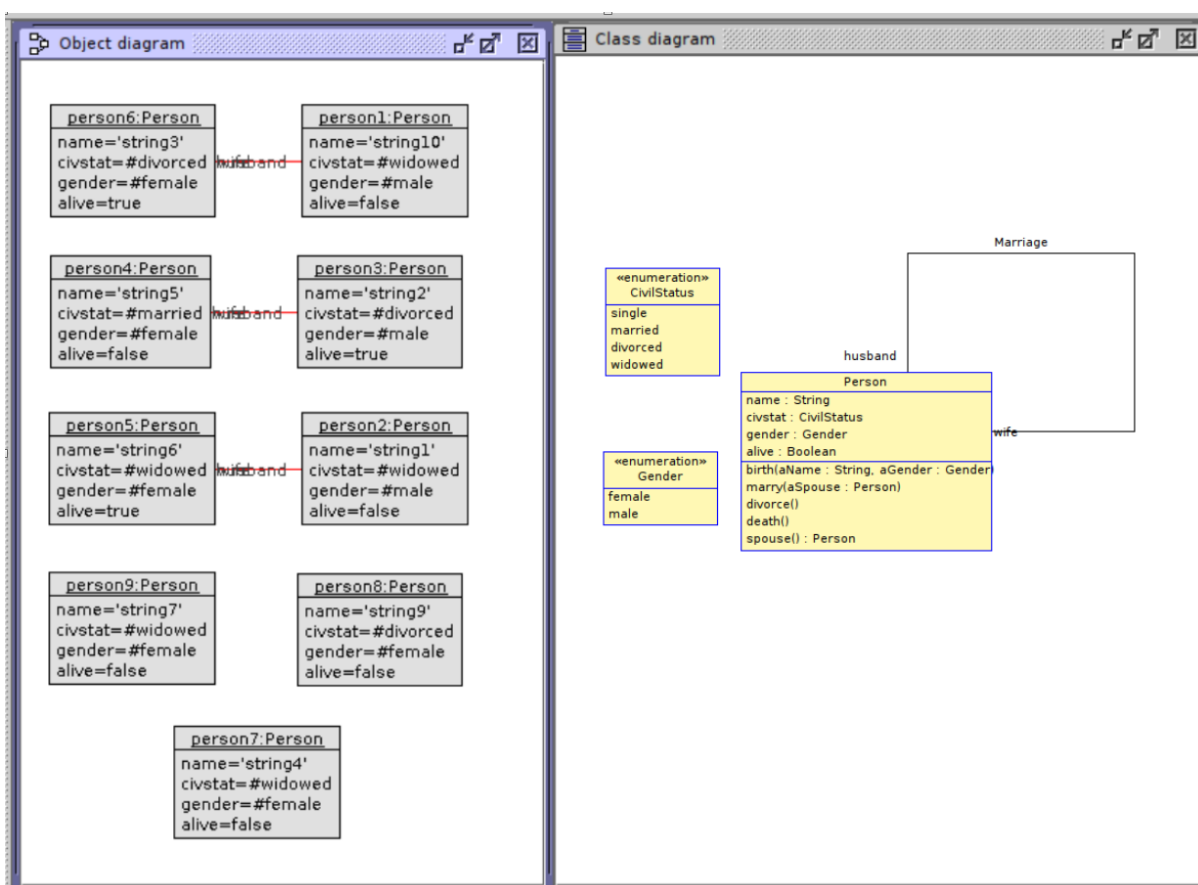
Hình 1.9 Giao diện công cụ USE [13].

1.3.4. Model Validator

Model Validator là một plugin được tích hợp sẵn trong công cụ USE, giúp người phát triển có thể tự động sinh các biểu đồ đối tượng cho một mô hình được định nghĩa trước. Các giá trị được sinh ra đều nằm trong khoảng giá trị được cấu hình, không gian tìm kiếm các giá trị được xác định thông qua một tệp cấu hình và các ràng buộc được mô tả trong mô hình gốc. Với mô hình ứng dụng được sử dụng làm đầu vào, model validator sẽ sử dụng một công cụ bên ngoài có tên là Kodkod để biểu diễn dưới dạng logic quan hệ. Logic quan hệ này sẽ được biểu diễn như một bài toán SAT (Boolean satisfiability problem) rồi được giải bằng các công cụ giải SAT. Nếu có lời giải trong không gian tìm kiếm được xác định từ đầu, model validator sẽ trả về kết quả SATISFIABLE. Nếu không có kết quả nào thỏa mãn toàn bộ các điều kiện đưa ra, kết quả UNSTISFIABLE sẽ được trả về. Kết quả đầu tiên thu được sẽ chuyển thành một

biểu đồ đối tượng được biểu diễn trên công cụ USE, và người dùng có thể tiếp tục tìm kiếm các kết quả khác để kiểm tra và xác nhận mô hình.

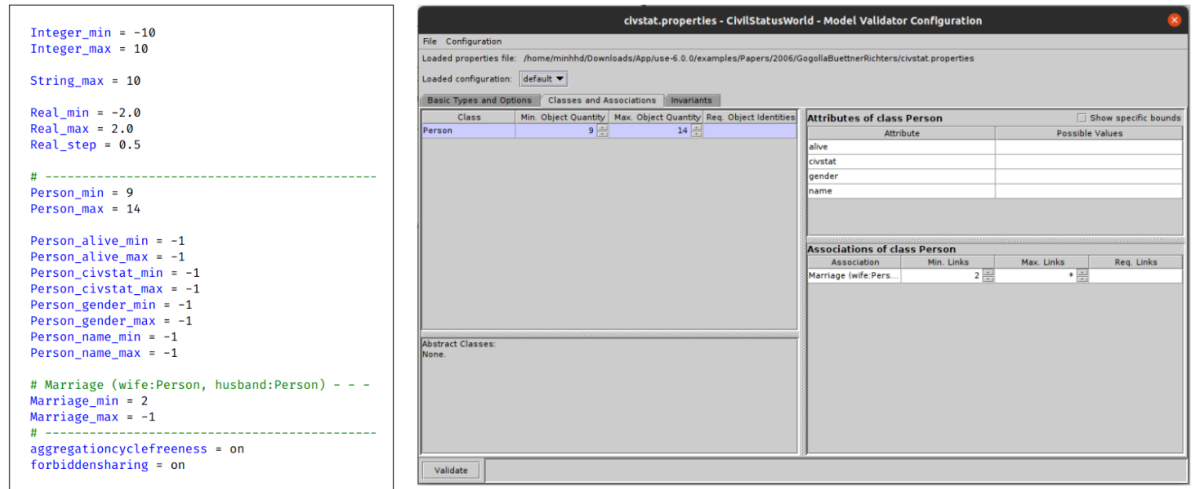
Tập cấu hình để xác định không gian tìm kiếm được thể hiện dưới dạng văn bản với đuôi .properties. Trong tập cấu hình này, người dùng cần xác định cận trên và cận dưới cho các kiểu giá trị nguyên thủy sẽ được sinh ra (Integer, Real, String), xác định số phần tử của mỗi lớp, mỗi thuộc tính hay số liên kết tương ứng. Ngoài ra, ta có thể xác định các giá trị có thể sử dụng với đối tượng đó. Việc cấu hình này giúp tăng tốc độ tìm kiếm mô hình, giới hạn phạm vi tìm kiếm khiến cho việc giải được mô hình thỏa mãn các ràng buộc trở nên dễ dàng hơn. Để làm rõ hơn quá trình thực hiện tìm kiếm mô hình tự động, ta có thể tham khảo ví dụ sau đây.



Hình 1.10 Biểu đồ đối tượng và biểu đồ lớp của mô hình Person.

Hình 1.10 mô tả một mô hình với lớp Person cùng quan hệ Marriage liên kết hai đối tượng của lớp Person. Dựa vào các thuộc tính và các bất biến lớp trong mô hình trên, người dùng cần thực hiện cấu hình để sinh dữ liệu bằng plugin Model Validator. Tập cấu hình được thể hiện trong Hình 1.11 với các thông tin về số lượng đối tượng được sinh, khoảng giá trị lớn nhất và nhỏ nhất của các thuộc tính trong các

đối tượng thuộc mô hình,... Kết quả thu được là một biểu đồ đối tượng, mô tả thông tin các đối tượng cùng các thuộc tính và quan hệ được sinh ra.



Hình 1.11 Tập cấu hình sinh dữ liệu và giao diện của plugin Model Validator.

1.4. Biểu diễn mô hình kịch bản với Filmstrip

Filmstrip là một plugin của công cụ USE được sử dụng để xác nhận và đánh giá hành vi của mô hình UML với các ràng buộc OCL. Nội dung phần này sẽ cung cấp kiến thức về plugin filmstrip cũng như mô hình filmstrip, ý tưởng và cách thức sử dụng plugin filmstrip để có thể kiểm thử hành vi của mô hình một cách tự động dựa vào các ràng buộc tiền và hậu điều kiện của hành vi của các đối tượng trong mô hình. Ngoài ra, phương pháp để xây dựng hậu điều kiện đủ chặt chẽ để ứng dụng kỹ thuật filmstrip sẽ được giới thiệu ở mục cuối của phần này.

Ngày nay, UML đã trở thành ngôn ngữ chung cho việc mô hình hóa các hệ thống công nghệ. Chính vì vậy, các kỹ thuật và công nghệ phục vụ cho mục đích xác nhận và đánh giá mô hình liên tục được phát triển và cải tiến, một trong số đó có thể kể đến là ModelValidator đã được trình bày ở phần trước. Tuy nhiên, các kỹ thuật này hầu như chỉ tập trung tới kiểm thử và đánh giá các thành phần tĩnh của hệ thống, liên quan đến cấu trúc và ràng buộc ví dụ như mô hình lớp hoặc các điều kiện OCL. Trong khi đó, một mô hình UML có thể bao gồm cả các thành phần động như hành vi của các đối tượng, được thể hiện dưới dạng các tiền và hậu điều kiện của các hành động. Mỗi lời gọi tới phương thức sẽ dẫn đến sự thay đổi trạng thái của các đối tượng và những thay đổi này không thể được phân tích bởi các công cụ đánh giá thành phần tĩnh của hệ thống.

Để xác nhận các khía cạnh động của mô hình, một phương pháp với tên gọi đã được phát triển và tích hợp vào công cụ USE. Ý tưởng của kỹ thuật này là theo dõi trạng thái của hệ thống thể hiện bằng các ảnh chụp hệ thống (snapshot), biểu diễn chuỗi ảnh chụp trạng thái này thành một trạng thái của một mô hình mới có tên gọi là mô hình filmstrip. Từ đó, việc đánh giá các hành vi của hệ thống sẽ được chuyển thành xác nhận và đánh giá mô hình filmstrip, giải quyết được bài toán đánh giá hành vi của mô hình UML.

1.4.1. Ý tưởng

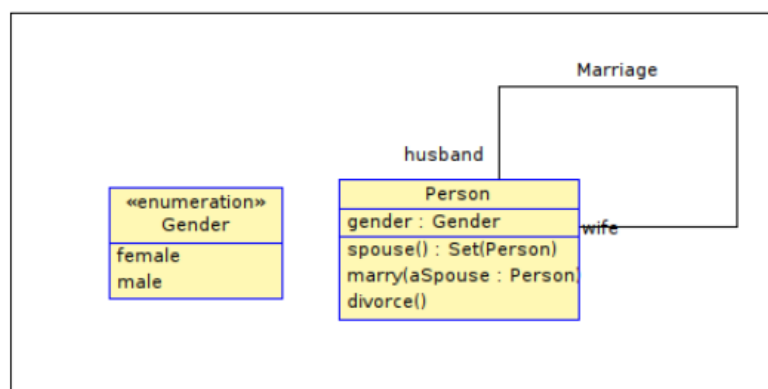
Mô hình ứng dụng. Mô hình ứng dụng là mô hình của hệ thống cần được theo dõi, đây là một mô hình UML bình thường bao gồm biểu đồ lớp với các lớp, thuộc tính, quan hệ cùng các phương thức. Ngoài ra, mô hình này cần được mô tả chi tiết hơn thông qua các ràng buộc OCL dưới dạng bất biến của lớp (class invariant) và tiền/hậu điều kiện của các phương thức. Các bất biến giới hạn các trạng thái mà hệ thống có thể thỏa mãn, trong khi đó, tiền và hậu điều kiện của phương thức xác định những thay đổi của hệ thống dựa vào sự chuyển trạng thái. Ở đây, sự chuyển trạng thái sẽ xảy ra sau mỗi lần gọi phương thức của một đối tượng. Việc xác định mô hình ứng dụng nhằm thể hiện toàn bộ ứng dụng sẽ được mô tả trong mô hình này, đồng thời phân biệt nó với mô hình filmstrip được mô tả bên dưới. Cần lưu ý, tất cả các thành phần kể trên của mô hình ứng dụng cần được xác định một cách đầy đủ và chặt chẽ, việc mô tả không đầy đủ mô hình ứng dụng như thiếu các thành phần hay các ràng buộc không chặt chẽ có thể khiến mô hình filmstrip được sinh ra không thỏa mãn được các yêu cầu cần thiết để kiểm thử hành vi của mô hình ứng dụng.

Mô hình filmstrip. Mô hình filmstrip mô tả chuỗi trạng thái của một mô hình ứng dụng dưới dạng một biểu đồ đối tượng: một tập các biểu đồ đối tượng tương ứng với mỗi trạng thái của mô hình ứng dụng cùng với các lời gọi phương thức để chuyển trạng thái được gói gọn thành một biểu đồ đối tượng của mô hình filmstrip. Với mỗi trạng thái của mô hình ứng dụng, một đối tượng có kiểu lớp Snapshot được sinh ra tương ứng với trạng thái đó. Lớp Snapshot là lớp được định nghĩa trong mô hình filmstrip, có tác dụng mô tả chuỗi trạng thái của mô hình ứng dụng thành một chuỗi các snapshot. Các đối tượng trong mô hình ứng dụng trong từng trạng thái tương ứng với các đối tượng thuộc một snapshot. Lời gọi phương thức trong mô hình ứng dụng sẽ trở thành một đối tượng phương thức trong mô hình filmstrip, có tác dụng liên kết các snapshot với nhau thành một chuỗi nhằm mô tả sự chuyển trạng thái. Nói tóm lại, một

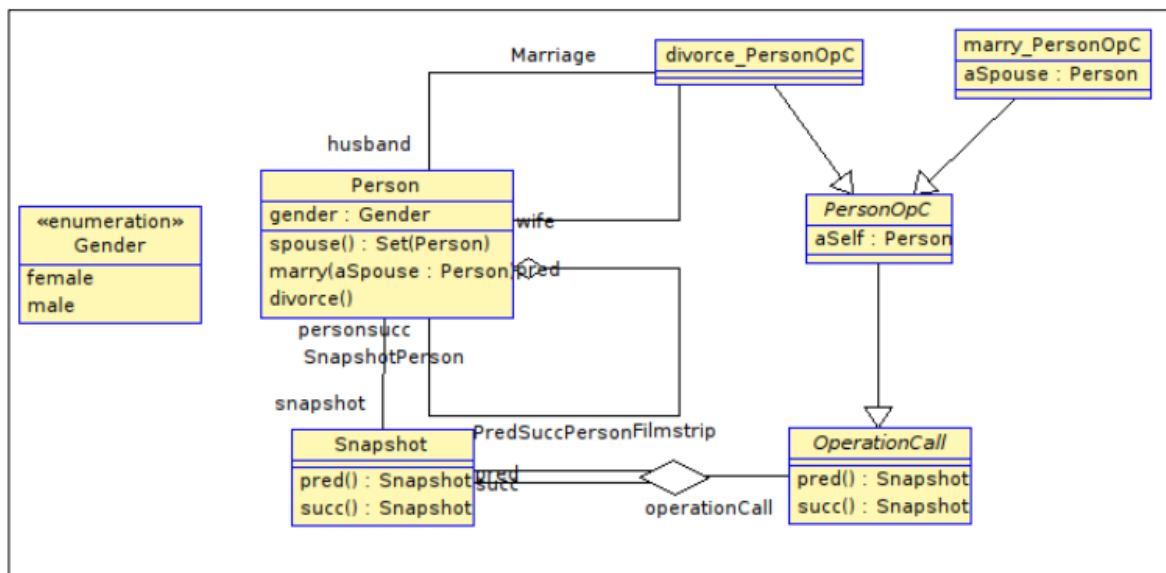
biểu đồ tuần tự của mô hình ứng dụng được thể hiện dưới dạng một biểu đồ đối tượng trong mô hình filmstrip.

1.4.2. Chuyển đổi từ mô hình ứng dụng sang mô hình filmstrip

Ví dụ về mô hình Person sẽ làm rõ hơn về quá trình chuyển đổi mô hình ứng dụng sang mô hình filmstrip. Trong ví dụ này, một đối tượng thuộc lớp Person có các phương thức như marry() và divorce(). Hình 1.12 biểu diễn biểu đồ lớp của mô hình ứng dụng bằng công cụ USE, bao gồm lớp Person cùng với quan hệ Marriage.



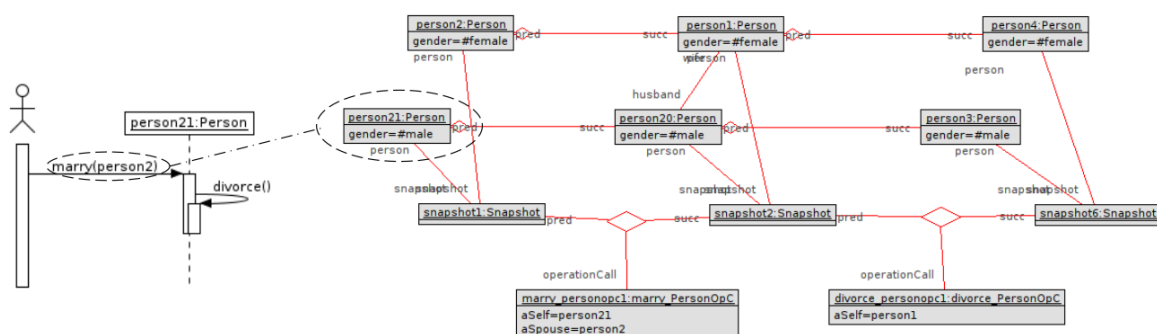
Hình 1.12 Biểu đồ lớp của mô hình ứng dụng trên công cụ USE.



Hình 1.13 Biểu đồ lớp của mô hình filmstrip.

Mô hình ứng dụng sẽ được chuyển đổi tự động sang mô hình filmstrip thông qua plugin được tích hợp với công cụ USE: thêm vào một số lớp và bất biến lớp cùng chuyển đổi các tiền/hậu điều kiện của phương thức và một số bất biến lớp ở mô hình

ứng dụng sang các bất biến lớp mới ở mô hình filmstrip. Biểu đồ lớp của mô hình filmstrip được sinh ra thể hiện trong Hình 1.13.



Hình 1.14 Mối tương quan giữa biểu đồ tuần tự của mô hình ứng dụng và biểu đồ đối tượng của mô hình filmstrip.

Cụ thể hơn, lớp Snapshot được thêm mới với vai trò mỗi đối tượng Snapshot sẽ thể hiện một trạng thái của của mô hình. Các phương thức trong mô hình ứng dụng tạo ra các lớp Operation tương ứng, với tiền tố OpC, mỗi đối tượng của lớp Operation này thể hiện một lời gọi phương thức tới đối tượng trong mô hình ứng dụng - điều này được thể hiện rõ hơn trong biểu đồ tuần tự ở Hình 1.14. Đối tượng OperationCall này có thuộc tính aSelf có giá trị là đối tượng được gọi phương thức, các tham số của phương thức cũng được chuyển thành các thuộc tính tương ứng. Ở ví dụ trên, lời gọi marry(person2) của đối tượng person21 trong biểu đồ tuần tự của mô hình ứng dụng được thể hiện trong biểu đồ đối tượng của mô hình filmstrip bằng đối tượng marry_personopcl với các thuộc tính aSelf là person21, aSpouse là person2. Lời gọi phương thức này đã chuyển trạng thái được thể hiện từ snapshot1 sang trạng thái ở snapshot 2, lúc này đối tượng person21 và person2 trong snapshot1 tương ứng với đối tượng person20 và person1 trong snapshot2, giữa hai đối tượng person20 và person1 đã có liên kết marriage biểu diễn việc gọi phương thức thành công.

Quá trình chuyển đổi từ mô hình ứng dụng sang mô hình filmstrip được thể hiện trong Hình 1.15. Ở phía tay trái là các thành phần của mô hình ứng dụng, bên trái là các thành phần tương ứng của mô hình filmstrip còn ở giữa của hình thể hiện quan hệ giữa các thành phần của nguồn là mô hình ứng dụng và đích là mô hình filmstrip. Mô tả chi tiết hơn về quá trình này được thể hiện như sau:

Application model		Filmstrip model
class	→ 1 : 1 →	application class
	*	class Snapshot
attribute	→ 1 : 1 →	attribute
operation (no return value)	→ Δ →	operation call class
operation self object	→ Δ →	operation call class attribute
operation parameter	→ Δ →	operation call class attribute
operation (with return value)	→ 1 : 1 →	operation in application class
association	→ 1 : 1 →	application association
	*	composition (Snapshot, application class)
	*	composition (Snapshot, operation call class)
	*	composition (operation call class, Snapshot)
	*	aggregation (application class, application class)
operation definition	→ 1 : 1 →	operation definition
class invariant	→ 1 : 1 →	application class invariant
	*	operation self object and parameter invariants
	*	filmstrip invariants
operation precondition	→ Δ →	operation call class invariant
operation postcondition	→ Δ →	operation call class invariant

Symbol explanation:	→ 1 : 1 →	model element is included without changes
	*	new model element is created
	→ Δ →	model element is included with changes applied

Hình 1.15 Các luật chuyển đổi từ mô hình ứng dụng sang mô hình filmstrip [11].

Chuyển đổi các lớp trong mô hình ứng dụng. Với tất cả các lớp cùng thuộc tính của mô hình ứng dụng sẽ được giữ nguyên thành các lớp và thuộc tính trong mô hình filmstrip. Ngoài ra, hai lớp Snapshot và OperationCall được thêm mới trong mô hình filmstrip. Lớp Snapshot sẽ gắn mỗi đối tượng với một snapshot tượng trưng cho một trạng thái của mô hình ứng dụng. Lớp OperationCall tượng trưng cho các lời gọi phương thức đại diện cho sự chuyển tiếp giữa các snapshot. Các phương thức của một lớp trong mô hình ứng dụng sẽ được chuyển thành một lớp trừu tượng kế thừa lớp OperationCall, có hậu tố OpC. Mỗi lời gọi phương thức sẽ tạo ra lớp con kế thừa lớp phương thức nói trên, có hậu tố C (tương ứng với chữ Call thể hiện đây là lời gọi hàm). Các lớp có kiểu trả về sẽ được giữ nguyên vì không tạo ra thay đổi trạng thái. Các tham số của phương thức trở thành các thuộc tính của lớp OperationCall tương ứng.

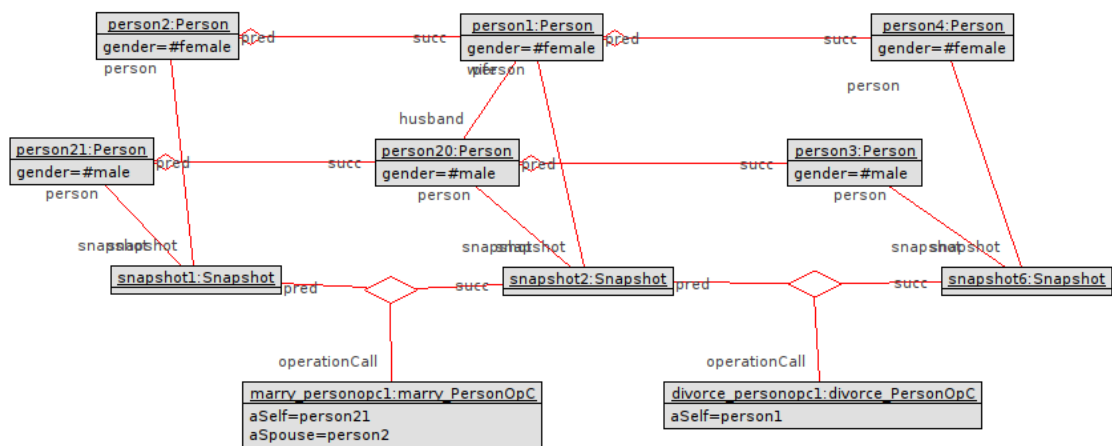
Ví dụ. Hình 1.13 thể hiện quan hệ kế thừa giữa các lớp OperationCall. Hai lớp *marryC* và *divorC* kế thừa lớp *PersonOpC* - lớp này lại kế thừa lớp *OperationCall*. *Operation spouse()* có giá trị trả về được giữ nguyên trong lớp *Person*.

Chuyển đổi các quan hệ trong mô hình ứng dụng. Tất cả quan hệ trong mô hình ứng dụng được giữ nguyên trong mô hình filmstrip. Ngoài ra, một số các quan hệ mới được thêm vào thể hiện mối quan hệ cho các lớp mới trong mô hình filmstrip. Quan hệ Filmstrip trong Hình 1.13 thể hiện cho sự chuyển trạng thái, được liên kết bởi hai đối tượng Snapshot tương ứng cho snapshot trước và sau lời gọi phương thức, cùng với một đối tượng OperationCall tương ứng với lời gọi phương thức. Quan hệ tiếp theo có tên với tiền tố Snapshot kết hợp với tên của một lớp trong mô hình ứng dụng (ví dụ như SnapshotPerson trong Hình 1.13) liên kết các lớp cùng với lớp Snapshot, thể hiện mỗi đối tượng trong mô hình ứng dụng trong một snapshot. Mỗi đối tượng này chỉ được liên kết với duy nhất một snapshot tương ứng, đồng thời liên kết với các đối tượng tương ứng ở các snapshot trước và sau thông qua quan hệ có tên với tiền tố PredSucc kèm tên lớp (ví dụ như PredSuccPerson trong Hình 1.13). Đối tượng đóng vai trò pred là đối tượng trước khi xảy ra chuyển trạng thái (thuộc snapshot trước khi chuyển trạng thái), tương ứng là đối tượng succ biểu diễn đối tượng của mô hình ứng dụng sau khi chuyển trạng thái.

Chuyển đổi định nghĩa phương thức và bất biến lớp. Định nghĩa phương thức sẽ được giữ nguyên khi chuyển sang mô hình filmstrip, còn các bất biến lớp sẽ được giữ nguyên hoặc thay đổi sao cho có ý nghĩa không đổi khi chuyển từ mô hình ứng dụng sang mô hình filmstrip.

Chuyển đổi các tiền và hậu điều kiện. Tiền và hậu điều kiện của các phương thức trong mô hình ứng dụng sẽ được chuyển thành các bất biến lớp trong mô hình filmstrip và được gán cho các lớp cụ thể kế thừa OperationCall. Các bất biến lớp này sẽ được kiểm tra đối với mỗi đối tượng OperationCall được tạo ra, tiền điều kiện sẽ được kiểm tra với snapshot có vai trò pred, hậu điều kiện kiểm tra với snapshot có vai trò succ. Điều này tương đương với việc kiểm tra tiền và hậu điều kiện mỗi khi gọi phương thức trong mô hình ứng dụng.

Sau khi đã chuyển đổi mô hình ứng dụng sang mô hình filmstrip, các thành phần động của mô hình ban đầu thể hiện các hành vi của mô hình đã được chuyển thành các thành phần tĩnh (các lời gọi phương thức chuyển thành các lớp OperationCall, tiền và hậu điều kiện chuyển thành các bất biến lớp,...). Khi đó, ta có thể sử dụng plugin Model Validator để sinh giá trị cho mô hình filmstrip. Kết quả thu được sẽ được thể hiện dưới dạng biểu đồ đối tượng như trong Hình 1.16.



Hình 1.16 Kết quả sau khi thực hiện tìm kiếm mô hình bằng Model Validator.

1.4.3 Phương pháp tạo ràng buộc khung PCDL

Trong một mô hình UML bình thường, OCL là ngôn ngữ khai báo [2] nên các ràng buộc OCL tiền và hậu điều kiện mô tả chức năng của một phương thức theo cách khai báo. Những ràng buộc này giới hạn trạng thái của hệ thống khi một phương thức được thực hiện và mô tả trạng thái mà hệ thống cần đạt được, bao gồm sự thay đổi của các đối tượng, các thuộc tính và các quan hệ. Tuy nhiên, tiền và hậu điều kiện chỉ tập trung đến các thành phần được tác động trong quá trình thực thi phương thức và thường bỏ qua các thành phần còn lại của hệ thống [3]. Vậy nên những ràng buộc này thường không thể mô tả một cách toàn diện bao gồm cả những thành phần thay đổi hay không đổi trong quá trình chuyển đổi trạng thái của hệ thống và từ đó dẫn đến các hành vi bất thường của một phương thức. Đồng thời, việc xác định những thành phần không được đề cập trong tiền và hậu điều kiện của phương thức rất quan trọng cho quá trình xác minh và xác nhận hệ thống (các điều kiện của các thành phần này thường được ngầm định trong quá trình thực thi phương thức).

Vấn đề về việc mô tả toàn diện hệ thống trong quá trình thay đổi trạng thái của hệ thống thường được giải quyết bằng việc thêm một số ràng buộc ngoài tiền và hậu điều kiện cho các phương thức, được gọi là ràng buộc khung (frame condition). Ràng buộc khung được đề cập trong nhiều lĩnh vực nghiên cứu khác nhau trong nhiều thập kỉ - ngoại trừ UML/OCL, các phương pháp tiếp cận với ràng buộc khung trong UML/OCL mới chỉ được phát triển trong những năm gần đây [4]. Quá trình tạo nên ràng buộc khung cho mô hình UML và OCL thường rất phức tạp, bởi một mô hình UML thường có một số lượng lớn các thành phần, đối tượng cũng như mối quan hệ

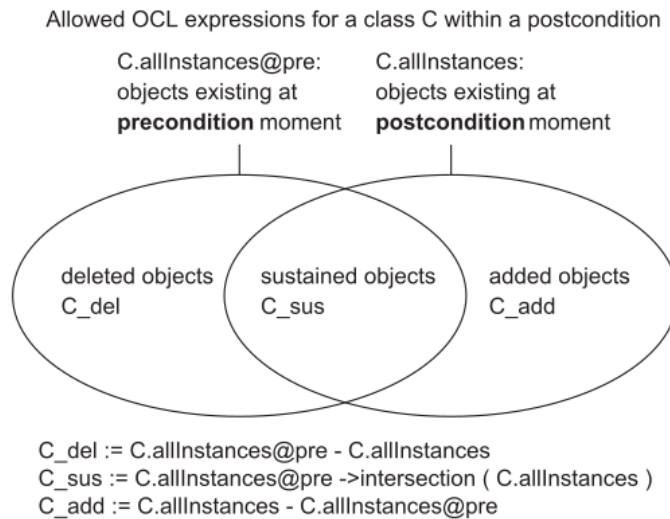
giữa các đối tượng đó. Đồng thời, việc xác định ràng buộc khung được thực hiện thủ công [5] dẫn tới việc tạo ra những ràng buộc thiếu chính xác.

Phương pháp có tên PCDL (post-/frame condition determining language) được đưa ra để giải quyết cho vấn đề này được đề cập trong báo cáo khoa học [6]. Đây là một phương pháp giúp người dùng có thể xây dựng ràng buộc khung một cách có hệ thống từ hậu điều kiện của phương thức, lưu ý tiền điều kiện không được đề cập trong phương pháp này. Theo hướng tiếp cận của phương pháp này, các thành phần trong mô hình PCDL sẽ theo dõi các hành vi thêm mới, loại bỏ và giữ nguyên của từng đối tượng trong mô hình ứng dụng ở mỗi phương thức, giúp đơn giản hóa việc đặc tả hậu điều kiện của phương thức. Từ đó, người dùng chỉ việc xác định các đối tượng sẽ chịu tác động của mỗi phương thức như thế nào, và nhóm các đối tượng đó theo các thành phần của PCDL và xác định được chi tiết sự thay đổi/không thay đổi của từng đối tượng. Sau khi xây dựng được mô hình dạng bảng của PCDL, người dùng có thể chuyển dạng mô hình đó sang hậu điều kiện ở định dạng OCL. Mục đích của phương pháp này nhằm xây dựng một “công thức” giúp giảm thiểu công sức của người thiết kế khi xây dựng ràng buộc khung, đồng thời mở ra một hướng mới để giải quyết bài toán tự động sinh ràng buộc khung cho mô hình UML/OCL.

1.4.3.1. Phân biệt các hành vi thêm mới, loại bỏ và giữ nguyên đối tượng

Ràng buộc khung cho một phương thức cần xác định được các thành phần thay đổi và những thành phần được giữ cố định trong quá trình thực thi một lời gọi phương thức. Trong một vài trường hợp, hậu điều kiện của phương thức còn mô tả các đối tượng được thêm mới hoặc loại bỏ đi sau mỗi phương thức. Để xác định rõ ràng hơn về toàn bộ đối tượng trong quá trình thực hiện phương thức, ta sẽ chia các trạng thái của các đối tượng sau khi thực hiện một phương thức ra làm ba trạng thái: *del* (delete - loại bỏ), *add* (thêm mới) và *sus* (sustain - giữ nguyên). Trong PCDL, quá trình loại bỏ hoặc thêm mới đi các đối tượng của một lớp sẽ được thể hiện ở trạng thái *del* và *add* của lớp đó. Một đối tượng không biến mất hay thêm mới sẽ có trạng thái *sus*, ngay cả khi thay đổi các liên kết hoặc thuộc tính của đối tượng đó. Hình 1.17 mô tả ý tưởng về các khái niệm được đề cập trên. Các đối tượng tồn tại trước khi thực thi phương thức sẽ nằm ở vòng tròn thứ nhất, các đối tượng tồn tại sau quá trình thực thi phương thức sẽ nằm ở vòng tròn thứ hai. Tương ứng với ba trạng thái, các đối tượng xuất hiện ở khu vực màu vàng là các đối tượng bị loại bỏ trong quá trình thực hiện phương thức, chỉ có thể xuất hiện ở trước phương thức mà không thể xuất hiện ở sau phương thức. Trong khi đó, các đối tượng thuộc khu vực màu xanh là các đối tượng được thêm mới,

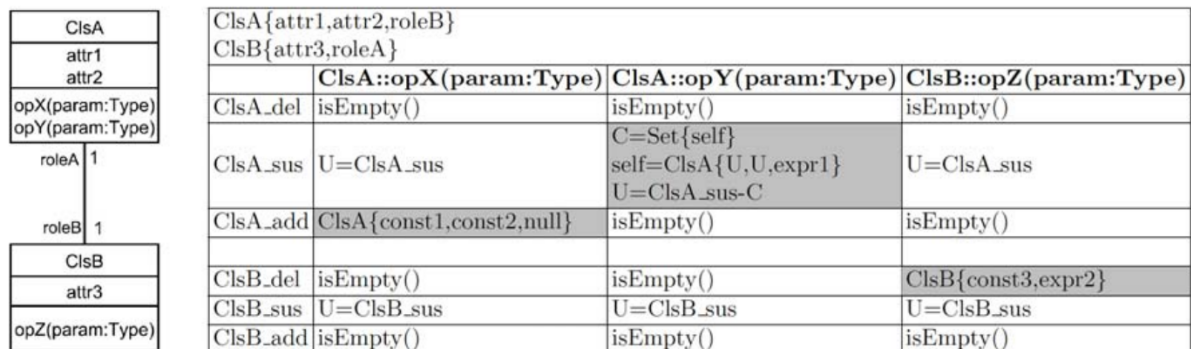
chỉ có thể xuất hiện ở sau phương thức mà không thể xuất hiện ở trước phương thức. Phần giao nhau ở giữa được giữ nguyên, có thể xuất hiện ở cả trước và sau phương thức. Các đối tượng trong cả ba phần trên đều có thể xác định một cách dễ dàng bằng các lệnh truy vấn OCL được thực hiện trong hậu điều kiện của phương thức. Việc xác định rõ ràng các thành phần trên có thể làm cơ sở để xây dựng ràng buộc khung một cách có hệ thống vào bao quát được toàn bộ đối tượng của hệ thống.



Hình 1.17 Miền của các đối tượng được thêm mới, loại bỏ và giữ nguyên trong quá trình thực hiện phương thức [6].

1.4.3.2. Biểu diễn hậu điều kiện của phương thức dưới dạng bảng PCDL

Từ các khái niệm đã được xây dựng ở trên, mô hình dạng bảng sẽ được xây dựng từ hành vi mong muốn của các phương thức được mô tả trong mô hình ứng dụng. Các phương thức này sẽ được chuyển thành các cột và ba trạng thái *del*, *add*, *sus* của mỗi lớp sẽ được chuyển thành các dòng. Một ví dụ tổng quát sẽ được mô tả trong Hình 1.18 để làm rõ hơn các giá trị của mô hình dạng bảng PCDL.



Hình 1.18 Ví dụ tổng quát. Biểu đồ lớp (bên trái) và biểu diễn dạng bảng của mô hình PCDL (bên phải) [6].

Ví dụ trên mô tả một mô hình gồm hai lớp ClsA và ClsB cùng với một quan hệ. Các thông tin về thuộc tính và quan hệ của cả hai lớp được thể hiện dưới dạng như $\text{ClsA}\{\text{attr1}, \text{attr2}, \text{roleB}\}$. Trong mô hình dạng bảng PCDL của ví dụ trên, các phương thức opX, opY và opZ được chuyển thành các cột và ba trạng thái của hai lớp ClsA và ClsB được chuyển thành các dòng tương ứng. Từ khóa U (unchange) và C (change) được sử dụng trong bảng để thể hiện các giá trị được giữ nguyên và các giá trị được thay đổi. Từ khóa self có tác dụng thể hiện đối tượng thực hiện phương thức. Các ô được tô tối màu trong bảng trên thể hiện những hành vi mà phương thức sẽ tác động đến các đối tượng, những hành vi này được xác định qua những hậu điều kiện được tạo ra khi xây dựng mô hình ứng dụng nhằm thể hiện các kết quả mong muốn sau khi thực thi phương thức. Người thiết kế sẽ phải tuân theo cấu trúc của PCDL để viết các hậu điều kiện. Ví dụ như trong phương thức opX, hậu điều kiện thể hiện việc thêm mới đối tượng ClsS được biểu diễn bằng cách thêm giá trị vào dòng ClsA_add giá trị $\text{ClsA}\{\text{const1}, \text{const2}, \text{null}\}$ tương ứng với giá trị các thuộc tính attr1, attr2 và liên kết roleB của A. Riêng với thành phần *sus* của bảng, người dùng cần xác định những đối tượng nào thay đổi trong tập C và những đối tượng nào không đổi trong tập U một cách chi tiết. Những thành phần không được đề cập tới sẽ có các giá trị mặc định, nếu ở các đối tượng của trạng *del* và *add*, giá trị mặc định sẽ là *isEmpty()* thể hiện không có đối tượng nào thuộc lớp được thêm mới hay loại bỏ. Giá trị mặc định ở trạng thái *sus* thể hiện qua biểu thức $U = \text{Cls_sus}$, thể hiện không có đối tượng nào thay đổi giá trị. Từ đó, hậu điều kiện của mỗi phương thức sẽ được biểu diễn dưới dạng bảng PCDL một cách có nguyên tắc và hệ thống. Điều cần làm tiếp theo là chuyển đổi mô hình PCDL này sang ràng buộc khung được biểu diễn dưới dạng hậu điều kiện.

1.4.3.3. Chuyển đổi từ mô hình PCDL sang hậu điều kiện

Sau khi đã xây dựng được mô hình dạng bảng PCDL từ hậu điều kiện, việc cần làm là chuyển mô hình này sang ràng buộc khung, các ràng buộc khung này sẽ được thể hiện dưới dạng hậu điều kiện. Hình 1.19 thể hiện quy tắc chuyển đổi từ các giá trị trong bảng PCDL sang hậu điều kiện OCL của ví dụ trong Hình 1.18.

ClsA{attr1,role1}		
	PCDL elements	Generic OCL postconditions
ClsA_del	isEmpty()	ClsA_del->isEmpty()
	ClsA{const1,expr1}	ClsA_del->size()==1 and (let v=ClsA_del->any(true) in v.attr1=const1 and v.role1=expr1)
ClsA_sus	U=ClsA_sus	ClsA_sus->forall(v v.attr1=v.attr1@pre and v.role1=v.role1@pre)
	C=Set{self}	self.attr1=self.attr1@pre and self.role1=expr2 and
	self=ClsA{U,expr2}	(ClsA_sus-Set{self})->forall(v v.attr1=v.attr1@pre and v.role1=v.role1@pre)
ClsA_add	U=ClsA_sus-C	
	isEmpty()	ClsA_add->isEmpty()
ClsA_add	ClsA{const2,expr3}	ClsA_add->size()==1 and (let v=ClsA_add->any(true) in v.attr1=const2 and v.role1=expr3)

Hình 1.19 Phương thức tổng quát chuyển đổi từ giá trị trong bảng sang hậu điều kiện OCL [6].

Trên hình 1.19, phần được tô xám là các giá trị do người dùng định nghĩa và được chuyển đổi sang hậu điều kiện còn phần màu trắng là các giá trị mặc định. Theo như đã trình bày trong Mục 1.4.3.1, ta có thể xác định các đối tượng có trạng thái *del*, *sus*, *add* bằng các biểu thức OCL. Như trong ví dụ trên, các đối tượng của ClsA_del sẽ được thể hiện bằng truy vấn ClsA.allInstances@pre-ClsA.allInstances, ClsA_sus tương ứng với ClsA.allInstances@pre->intersection(ClsA.allInstances) và ClsA_add tương ứng với ClsA.allInstances-ClsA.allInstances@pre. Trong đó hậu tố @pre thể hiện giá trị trước khi thực hiện phương thức, còn hàm intersection sẽ trả về tập giao thoa giữa hai tập ClsA.allInstances và ClsA.allInstances@pre. Sau khi chuyển đổi từ các thành phần của PCDL sang hậu điều kiện và thay thế các giá trị tương ứng, ta thu được các ràng buộc khung mô tả được đầy đủ hành vi của mô hình UML và OCL. Cần lưu ý, xây dựng các ràng buộc khung này chỉ biến đổi hậu điều kiện gốc của phương thức để xác định hành vi của phương thức. Do đó các tiền điều kiện sẽ được giữ nguyên làm điều kiện để thực hiện phương thức.

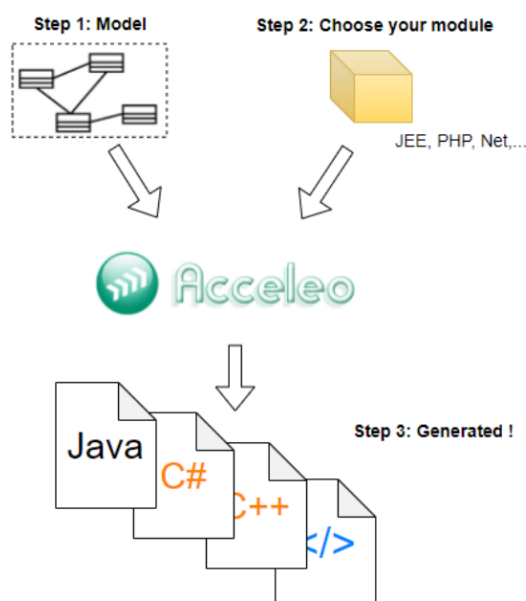
1.6. Một số công nghệ chuyển đổi mô hình

Chuyển đổi mô hình chính là trái tim và linh hồn của kỹ thuật phát triển phần mềm hướng mô hình [1]. Chuyển đổi mô hình là quá trình chuyển đổi mô hình sang các tạo tác phần mềm khác (như mã nguồn,v.v). Chuyển đổi mô hình sang văn bản là một trong những kỹ thuật chuyển đổi mô hình, với ứng dụng lớn nhất là sinh mã nguồn từ mô hình. Phần này sẽ giới thiệu về kỹ thuật chuyển đổi mô hình sang văn bản cùng công nghệ Acceleo sẽ được sử dụng trong khóa luận.

Acceleo là sản phẩm mã nguồn mở của nhiều năm nghiên cứu và phát triển của các nhóm triển lược Obeo của Eclipse. Acceleo là một công nghệ sinh mã nguồn phát triển từ đặc tả của OMG về tiêu chuẩn kỹ thuật trong chuyển đổi mô hình sang văn bản.

Acceleo cung cấp rất nhiều chức năng cần thiết của một môi trường phát triển tích hợp (IDE) sinh mã nguồn: cú pháp đơn giản, khả năng sinh mã nguồn hiệu quả cùng các tính năng hỗ trợ lập trình viên như xác định lỗi, đánh dấu cú pháp, tái cấu trúc v.v. Acceleo được phát triển tuân thủ nhiều tiêu chuẩn khác nhau, như EML, OCL, UML. Việc tuân thủ các tiêu chuẩn này giúp Acceleo có thể được tích hợp và sử dụng cùng với các công nghệ sử dụng EMF khác trong Eclipse như biểu diễn mô hình bằng đồ họa v.v, đồng thời sử dụng ngôn ngữ OCL làm ngôn ngữ truy vấn giúp quá trình phát triển dự án trở nên thuận tiện và dễ dàng hơn.

Quá trình sinh mã nguồn của Acceleo được thể hiện dưới Hình 1.20. Đầu vào cho quá trình sinh mã nguồn từ mô hình yêu cầu một mô hình nguồn và siêu mô hình của mô hình đó phục vụ cho việc truy vấn của Acceleo. Tiếp theo, người phát triển cần xây dựng một khuôn mẫu cho các thành phần của mã nguồn cần được sinh ra. Kết quả thu được sau khi được Acceleo thực thi là một tập các tệp được xây dựng thỏa mãn theo các khuôn mẫu, với giá trị được lấy ra từ các truy vấn tới mô hình nguồn.



Hình 1.20 Các bước sinh mã nguồn của Acceleo.

Hướng tiếp cận được đề ra trong quá trình xây dựng khuôn mẫu là phương pháp “bottom-up”. Phương pháp này bao gồm quá trình xây dựng các “bản mẫu” trước, sau đó thực hiện các bước kiểm thử về hiệu suất, quy tắc lập trình cũng như sự ổn định khi được cài đặt trong môi trường thực thi. Sau khi quá trình kiểm tra và thẩm định kết thúc, bản mẫu này sẽ trở thành căn cứ để người phát triển tạo nên các khuôn mẫu, từ

đó mã nguồn được sinh ra từ các khuôn mẫu sẽ tối ưu nhất, giảm thiểu chi phí bảo trì cũng như các vấn đề phát sinh.

1.7. Tổng kết chương

Các kiến thức nền tảng cùng cơ sở lý thuyết cần thiết cho khoa luận được giới thiệu trong chương 1. Những định nghĩa, kiến thức về ngôn ngữ mô hình thống nhất, ngôn ngữ ràng buộc đối tượng, ngôn ngữ đặc tả ca sử dụng FRSL là các cơ sở để phát triển phương pháp mà khóa luận hướng tới. Ngoài ra, các công nghệ được sử dụng cho khóa luận như công nghệ chuyển đổi mô hình sang văn bản Acceleo, công cụ USE với plugin filmstrip và Model Validator là những thành phần thiết yếu trong quá trình sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng.

CHƯƠNG 2. PHƯƠNG PHÁP SINH TỰ ĐỘNG DỮ LIỆU KIỂM THỬ TỪ ĐẶC TẢ CA SỬ DỤNG FRSL

Nội dung chương này tập trung mô tả phương pháp sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng FRSL, bao gồm cụ thể các bước thực hiện, các thành phần, công cụ tham gia vào quá trình này.

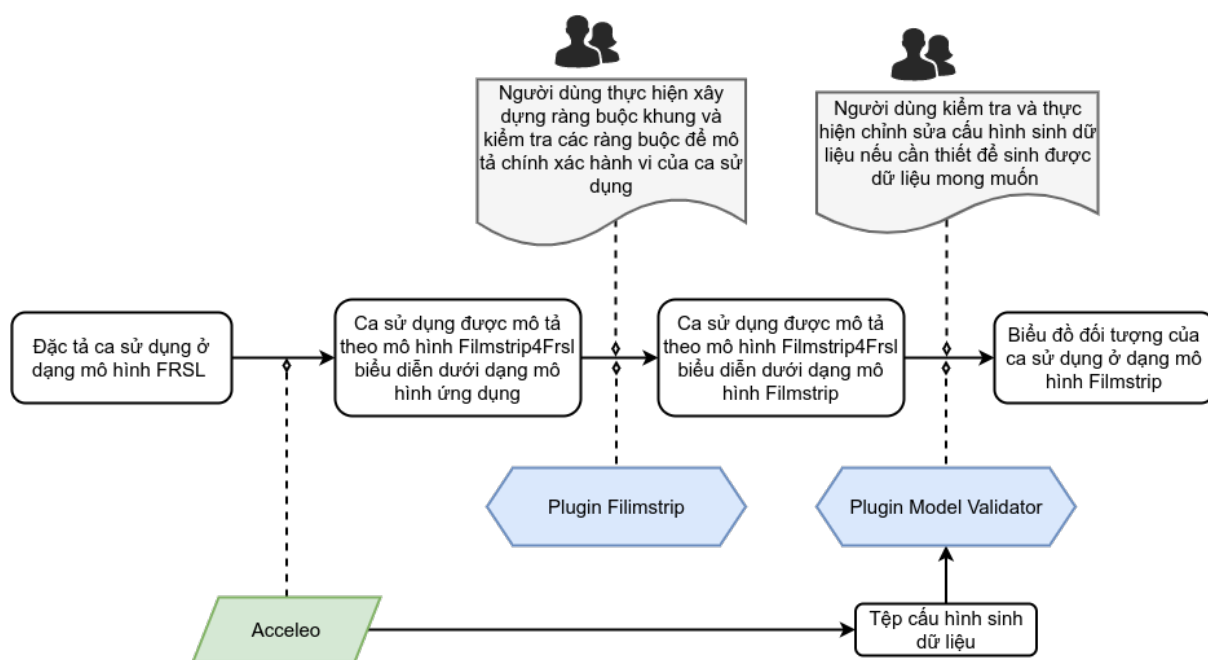
2.1. Giới thiệu

Mục đích của chương này là đưa ra giải pháp cho bài toán sinh tự động dữ liệu kiểm thử từ đặc tả FRSL. Phương pháp được sử dụng để giải quyết vấn đề sinh tự động là sử dụng các công nghệ được giới thiệu ở chương hai bao gồm plugin sinh dữ liệu tự động cho mô hình Model Validator và plugin Filmstrip với công dụng kiểm thử, xác nhận hành vi của mô hình UML. Tuy nhiên, vấn đề giải quyết tiếp theo là làm sao có thể biểu diễn đặc tả ca sử dụng ở dạng mô hình FRSL thành một mô hình UML với đầy đủ các điều kiện để có thể làm đầu vào cho plugin Filmstrip để có thể sinh tự động dữ liệu kiểm thử. Chính vì vậy, khóa luận đề xuất một mô hình trung gian có tên Filmstrip4Frsl, mô hình này sẽ được sinh tự động từ mô hình FRSL nhờ công cụ chuyển đổi mô hình Acceleo, đạt được đầy đủ các yêu cầu để làm đầu vào cho plugin filmstrip cũng như biểu diễn được hành vi của ca sử dụng.

Các phần tiếp theo của chương 3 sẽ được tổ chức theo cấu trúc sau. Mục 2.2 sẽ cung cấp quy trình tổng quan của phương pháp, các bước và các thành phần tham gia quá trình sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng. Mục 2.3 mô tả về mô hình được đề xuất Filmstrip4Frsl, các yêu cầu cũng như các thành phần của mô hình này. Luật chuyển đổi Acceleo để chuyển đổi mô hình FRSL sang mô hình Filmstrip4Frsl cùng các tạo tác sẽ được mô tả trong mục 2.4. Cuối cùng, mục 2.5. sẽ thể hiện cách thức sử dụng plugin filmstrip và model validator để sinh dữ liệu tự động cho đặc tả ca sử dụng.

2.2. Tổng quan về phương pháp

Phần này tập trung mô tả tổng quan quá trình sinh dữ liệu cho ca sử dụng, giới thiệu thành phần và các bước thực hiện. Để thực hiện sinh dữ liệu một cách tự động, khóa luận đề xuất phương pháp chuỗi chuyển đổi. Chuỗi chuyển đổi trên được thể hiện bằng hình vẽ 2.



Hình 2.1 Toàn bộ quy trình thực hiện phương pháp sinh tự động dữ liệu từ đặc tả ca sử dụng.

Bước 1: Quá trình sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng bắt đầu từ đặc tả ca sử dụng ở dạng mô hình FRSL. Mô hình FRSL này sẽ được chuyển tự động sang mô hình Filmstrip4Frsl cùng với tập cấu hình sinh dữ liệu theo các luật chuyển được mô tả trong Mục 2.4. Mô hình Filmstrip4Frsl được sinh ra sẽ biểu diễn dưới dạng mô hình ứng dụng và được thêm các ràng buộc khung để có thể làm đầu vào cho plugin filmstrip.

Bước 2: Tiếp theo, người dùng cần thực hiện xây dựng các ràng buộc khung bằng phương pháp PCDL được đề cập ở Mục 1.4.3 cùng với một số cải tiến được mô tả trong Mục 2.3.1.2. Sau đó, mô hình Filmstrip4Frsl biểu diễn dưới dạng mô hình ứng dụng sẽ được chuyển tự động sang mô hình Filmstrip4Frsl biểu diễn dưới dạng mô hình filmstrip dựa vào plugin filmstrip của công cụ USE. Ở dạng mô hình filmstrip, một biểu đồ đối tượng có thể mô tả một chuỗi trạng thái của hệ thống và các lời gọi phương thức ở giữa các trạng thái. Các thành phần động của mô hình Filmstrip4Frsl ở dạng mô hình ứng dụng sẽ được chuyển thành các thành phần tĩnh khi ở dạng mô hình filmstrip, từ đó có thể thực hiện sinh dữ liệu tự động cho các thành phần tĩnh đó. Cần lưu ý thêm, người dùng nên kiểm tra các ràng buộc sau khi được sinh ra trong quá trình chuyển đổi và chỉnh sửa nếu cần thiết để hệ thống thể hiện được các hành vi đúng đắn nhất.

Bước 3: Sử dụng tệp cấu hình được sinh ra từ bước 1, người dùng có thể cấu hình thêm để dữ liệu được sinh ra có giá trị như mong muốn. Tệp cấu hình sẽ được sử dụng cho plugin Model Validator để sinh dữ liệu, dữ liệu thu được sẽ được thể hiện dưới dạng biểu đồ đối tượng, mô tả các trạng thái của hệ thống trong một luồng của ca sử dụng.

2.3. Biểu diễn mô hình Filmstrip4Frsl

Phần này sẽ mô tả chi tiết về mô hình Filmstrip4Frsl được đề xuất, yêu cầu mà mô hình cần được thỏa mãn và các thành phần cũng như hai dạng biểu diễn của mô hình Filmstrip4Frsl. Phương pháp sinh dữ liệu cho ca sử dụng được dựa trên ý tưởng sử dụng plugin filmstrip. Công dụng của plugin này đã được giới thiệu trong chương 2, đó là kiểm tra và thẩm định hành vi của một mô hình bằng cách biểu diễn mô hình đó dưới dạng mô hình filmstrip, biến các thành phần động trong mô hình gốc thành các thành phần tĩnh trong mô hình filmstrip để từ đó có thể sinh dữ liệu.

Tuy nhiên, sử dụng plugin filmstrip để sinh tự động mô hình filmstrip yêu cầu một mô hình ứng dụng dưới dạng file có đuôi .use làm đầu vào cho việc chuyển đổi. Đặc tả cả sử dụng mô tả bằng ngôn ngữ FRSL có định dạng file xmi, đồng thời mục đích sử dụng chính của FRSL để mô tả các luồng trong ca sử dụng trong khi filmstrip mô tả các chuỗi thay đổi trạng thái của hệ thống dưới dạng một biểu đồ đối tượng [1]. Trong thực tế, trạng thái của một hệ thống có thể được tạo bởi nhiều ca sử dụng, nhiều luồng thực hiện cùng một lúc. Do đó ca sử dụng mô tả bằng FRSL không thể làm đầu vào trực tiếp cho plugin filmstrip. Để giải quyết bài toán trên, khóa luận đề xuất xây dựng một mô hình Filmstrip4Frsl làm trung gian hỗ trợ việc chuyển đổi mô hình ca sử dụng sang mô hình filmstrip để từ đó có thể sinh dữ liệu cho ca sử dụng.

2.3.1. Yêu cầu của Filmstrip4Frsl

Mô hình filmstrip là kết quả được tạo nên từ việc biến đổi mô hình ứng dụng (bao gồm các lớp và các mối quan hệ giữa các lớp) cùng với tiền và hậu điều kiện của các hành động có thể được thực hiện để hệ thống chuyển sang 1 trạng thái mới [2]. Vì vậy, để Filmstrip4Frsl có thể làm đầu vào cho việc chuyển đổi sang mô hình filmstrip, Filmstrip4Frsl cần mô tả được mô hình ứng dụng của một ca sử dụng và tiền/hậu điều kiện của các hành động trong ca sử dụng đó.

Tuy nhiên, như đã đề cập ở trên, mô hình filmstrip mô tả chuỗi các trạng thái của hệ thống, có thể đạt được qua nhiều luồng/ca sử dụng diễn ra cùng một lúc. Trong khi đó, một ca kiểm thử của một ca sử dụng chỉ tập trung vào duy nhất một luồng. Từ

đó, Filmstrip4Frsl phải được thiết kế sao cho mô hình filmstrip được sinh ra chỉ có thể đi theo một luồng duy nhất. Vấn đề trên được giải quyết bằng cách thêm các ràng buộc tiền và hậu điều kiện ngoài các điều kiện được mô tả trong FRSL từ trước, đồng thời thay đổi tệp cấu hình để dữ liệu được sinh ra thể hiện một chuỗi các trạng thái của hệ thống qua một luồng duy nhất.

2.3.2. Biểu diễn *Filmstrip4Frsl* dưới dạng mô hình ứng dụng

2.3.2.1. Các thành phần chính của *Filmstrip4Frsl* ở dạng mô hình ứng dụng

Theo kỹ thuật biểu diễn mô hình kịch bản Filmstrip, mô hình ứng dụng của hệ thống được sử dụng làm đầu vào để chuyển đổi sang mô hình filmstrip. Chính vì vậy, có thể coi *Filmstrip4Frsl* là mô hình ứng dụng trong kỹ thuật Filmstrip, phải có đầy đủ các thành phần cũng như thỏa mãn các điều kiện của mô hình ứng dụng đã được đề cập trong chương 1. Trong *Filmstrip4Frsl* ở dạng mô hình ứng dụng, có thể chia làm hai thành phần với thành phần đầu tiên được mô tả biểu đồ lớp của các đối tượng trong ca sử dụng mô tả. Thành phần còn lại là các bất biến lớp và tiền/hậu điều kiện được thêm vào nhằm mục đích mô tả hành vi của ca sử dụng. Các bất biến lớp và tiền/hậu điều kiện là các thành phần được sinh bán tự động và cần người dùng can thiệp nên sẽ được trình bày ở Mục 2.3.1.2.

<pre> package ticketModel{ class Ticket{ attribute value: Integer; } class Customer { attribute money: Integer; } } association CustomerTicket ticket: Ticket[0..1] customer: Customer[0..1] end actor Customer end </pre>	<pre> usecase BuyTicket description = 'This use-case describes customer buying ticket' primaryActor = Customer ucPrecondition description = 'Customer is ready to buy ticket.' customer: Customer; ticket: Ticket; [customer.money>0] [ticket.value>0] end ucPostcondition description = 'Customer buys ticket successfully.' customer: Customer; ticket: Ticket; (customer, ticket): CustomerTicket; end actStep step01 description = '1. Customer pays for the ticket' from customer: Customer; ticket: Ticket; [customer.money>ticket.value] to [customer.money@pre=customer.money+ticket.value] end actStep step02 description = '2. Customer takes that ticket' from customer: Customer; ticket: Ticket; to (customer, ticket): CustomerTicket; end end end </pre>
--	--

Hình 2.2 Ví dụ. Ca sử dụng *BuyTicket* biểu diễn bằng ngôn ngữ FRSL.

Một ví dụ minh họa về một ca sử dụng được dùng để làm rõ hơn các thành phần trong mô hình Filmstrip4Frsl được mô tả ở dạng văn bản bằng ngôn ngữ FRSL thể hiện trong Hình 2.2. Trong ví dụ này, có hai lớp tham gia ca sử dụng là lớp Ticket và lớp Customer, hai lớp liên kết với nhau bởi quan hệ CustomerTicket. Ca sử dụng BuyTicket gồm hai bước, bước đầu tiên người dùng sẽ thực hiện trả tiền vé và bước thứ hai người dùng sẽ lấy vé. Tại bước đầu tiên, hành vi trả tiền vé của đối tượng customer được thể hiện bằng ràng buộc OCL. Hành vi của các đối tượng được thể hiện bởi các snapshot (ảnh chụp hệ thống), trước và sau mỗi bước thực hiện. Với ví dụ minh họa trên, ta bắt đầu phân tích rõ hơn về thành phần đầu tiên của mô hình Filmstrip4Frsl biểu diễn ở dạng mô hình ứng dụng.

Biểu đồ lớp tham gia ca sử dụng. Các lớp tham gia ca sử dụng là các lớp được người dùng định nghĩa, đóng vai trò nhất định trong một ca sử dụng. Filmstrip4Frsl diễn tả các lớp và mối quan hệ của các lớp của mỗi ca sử dụng bằng cú pháp của được định nghĩa của công cụ USE.

Mỗi lớp tham gia ca sử dụng được mô tả trong Filmstrip4Frsl bao gồm các thông tin như tên, các thuộc tính của lớp đó. Thuộc tính của một lớp có thể là kiểu dữ liệu nguyên thủy hoặc là một lớp khác. Vì mô hình FRSL không mô tả các phương thức của các lớp này nên phương thức của các lớp tham gia ca sử dụng không được định nghĩa trong Filmstrip4Frsl.

```
class Customer
  attributes
    money: Integer
  end

class Ticket
  attributes
    value: Integer
  end

association CustomerTicket between
  Customer[0..1]
  Ticket[0..1]
end
```

Hình 2.3 Biểu diễn lớp tham gia ca sử dụng cùng các quan hệ của mô hình Filmstrip4Frsl bằng cú pháp của công cụ USE.

Quan hệ giữa các lớp tham gia ca sử dụng được giữ nguyên trong Filmstrip4Frsl. Tuy nhiên, cần lưu ý các quan hệ là có số lượng đối tượng tham gia liên kết của một lớp có cận dưới lớn hơn hoặc bằng 1, quan hệ này khẳng định tất cả

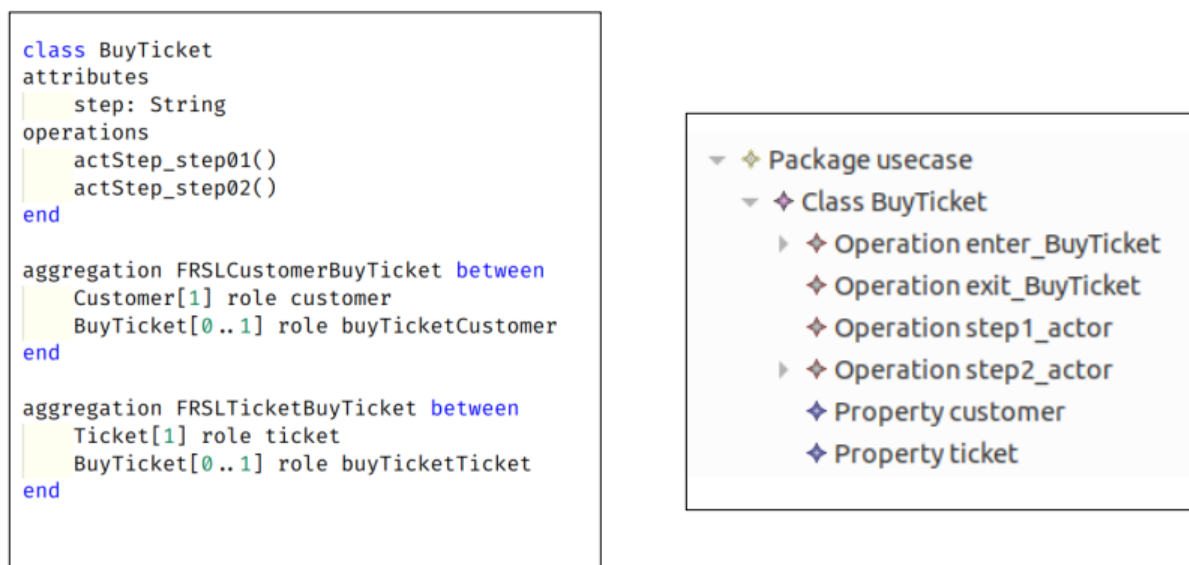
các đối tượng thuộc lớp còn lại đều phải tham gia quan hệ này. Ví dụ như quan hệ SalePayment trong Hình 2.4, số lượng đối tượng của lớp Sale tham gia liên kết tương ứng với mỗi đối tượng của lớp Payment là 1. Khi đó, toàn bộ đối tượng Payment ở mọi trạng thái đều phải tham gia liên kết. Điều này có thể hợp lý với biểu đồ lớp của các lớp tham gia ca sử dụng, nhưng lại không thích hợp trong mô hình Filmstrip4Frsl bởi các đối tượng ràng buộc có thể nằm ngoài phạm vi mô tả trong ca sử dụng. Ví dụ, trong ca sử dụng ProccessSale tại thời điểm khách hàng thanh toán, có thể đối tượng Sale được tạo ra nhưng Payment lại chưa được tạo. Chính vì vậy, người thiết kế cần kiểm tra thật kĩ các quan hệ giữa các lớp tham gia ca sử dụng để có thể sinh được các đối tượng thỏa mãn.

```

association CustomerTicket
    ticket: Ticket[1]
    customer: Customer[1]
end

```

Hình 2.4 Ví dụ về một quan hệ không hợp lệ trong Filmstrip4Frsl.

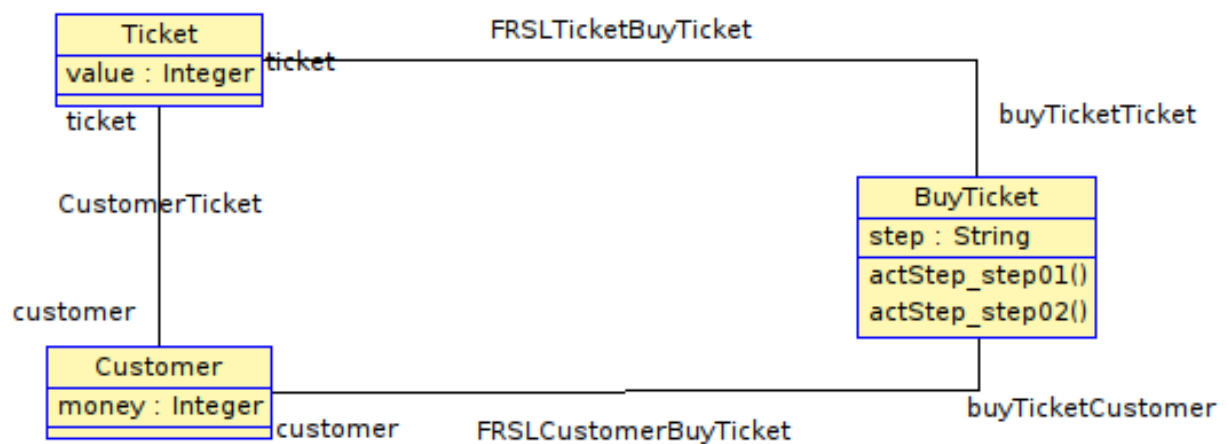


Hình 2.5 Lớp ca sử dụng và các quan hệ trong Filmstrip4Frsl (bên trái) và lớp ca sử dụng trong FRSL (bên phải).

Lớp ca sử dụng. Tương tự như mô hình FRSL, Filmstrip4Frsl cũng sử dụng lớp ca sử dụng để mô tả các hành vi và trạng thái của ca sử dụng. Khác với lớp ca sử dụng trong FRSL, Filmstrip4Frsl loại bỏ đi một số thuộc tính là các đối tượng tham gia vào ca sử dụng mà thay vào đó sử dụng quan hệ hợp thành (aggregation) có tiền tố FRSL để có thể áp dụng thêm một số ràng buộc, các ràng buộc này sẽ được mô tả chi tiết hơn trong Mục 2.3.1.2. Ví dụ như thuộc tính customer trong ca sử dụng được mô tả trong

FRSL, sẽ được chuyển thành quan hệ như trong Hình 2.5. Các thuộc tính kiểu nguyên thủy sẽ được giữ nguyên làm thuộc tính của ca sử dụng, ngoài ra, thêm mới thuộc tính step có kiểu chuỗi nhằm thể hiện tên bước cuối cùng được thực hiện để dẫn đến một trạng thái bất kì. Mỗi bước của ca sử dụng sẽ chuyển đổi trạng thái của hệ thống nên được thể hiện tương ứng thành một phương thức của lớp ca sử dụng.

Ví dụ. Trong mô hình Hình 2.5, lớp ca sử dụng BuyTicket có quan hệ hợp thành với tất cả các lớp tham gia ca sử dụng (Ticket, Customer), các thuộc tính nguyên thủy của ca sử dụng sẽ được giữ nguyên. Hai bước trong ca sử dụng BuyTicket được chuyển tương ứng thành hai phương thức của lớp BuyTicket.



Hình 2.6 Toàn bộ các lớp và quan hệ trong mô hình Filmtrip4Frsl ở dạng mô hình ứng dụng của ca sử dụng BuyTicket.

2.3.2.2. Bất biến lớp và tiền, hậu điều kiện cho các phương thức

Ngoài các thành phần lớp và quan hệ, Filmstrip cần thêm các bất biến lớp và tiền/hậu điều kiện cho các phương thức để mô tả hành vi trong ca sử dụng.

Bất biến lớp. Lớp ca sử dụng được mô tả trong Filmstrip4Frsl cần phải được thỏa mãn hai điều kiện. Đầu tiên, chỉ có duy nhất một đối tượng của lớp ca sử dụng trong một snapshot. Filmstrip4Frsl có vai trò biểu diễn hệ thống trong một luồng của một ca sử dụng, việc có nhiều ca sử dụng trong một snapshot là điều không hợp lý. Như ví dụ trong ca sử dụng BuyTicket, tại một thời điểm bất kì chỉ có duy nhất một

đối tượng của lớp BuyTicket được tồn tại. Điều kiện này được thể hiện bằng bất biến lớp như sau:

```
context BuyTicket inv usecaseConstraint:  
    BuyTicket.allInstances()→size=1
```

Hình 2.7 Bất biến lớp ràng buộc lớp BuyTicket chỉ có duy nhất 1 đối tượng.

Tiếp theo, mỗi đối tượng tham gia ca sử dụng đều phải có liên kết với lớp ca sử dụng để giúp việc quản lý các đối tượng trong ca sử dụng trở nên dễ dàng hơn. Thay vì phải diễn tả sự thay đổi của tất cả các đối tượng trong một trạng thái, ta chỉ cần tập trung đến lớp ca sử dụng. Trạng thái của các đối tượng sẽ được truy vấn dựa vào lớp ca sử dụng, từ đó đơn giản hóa quá trình mô tả ca sử dụng. Để thỏa mãn điều kiện trên, ta cần thiết lập ràng buộc sao cho: mỗi đối tượng đều phải có một và chỉ một liên kết với một đối tượng lớp ca sử dụng, đồng thời ca sử dụng phải tham gia vào tất cả các quan hệ đã xác định trước. Điều kiện này được thể hiện qua quan hệ giữa đối tượng đó và đối tượng lớp ca sử dụng có tiền tố FRSL cùng với bất biến lớp như Hình 2.8 dưới đây. Ràng buộc này thể hiện với mỗi đối tượng thuộc lớp Customer và Ticket, đối tượng bắt buộc phải có một liên kết với duy nhất với đối tượng thuộc lớp ca sử dụng BuyTicket. Đồng thời số lượng đối tượng tham gia liên kết với lớp ca sử dụng còn được thể hiện qua các quan hệ FRSLCustomerBuyTicket và FRSLTicketBuyTicket trong Hình 2.6, yêu cầu với mỗi đối tượng lớp ca sử dụng đều phải tham gia tất cả các liên kết trên.

```
context BuyTicket inv usecaseConnectAll:  
    Customer.allInstances→forall(x|x.buyTicketCustomer→size() = 1) and  
    Ticket.allInstances→forall(x|x.buyTicketTicket→size() = 1)
```

Hình 2.8 Bất biến lớp ràng buộc quan hệ giữa lớp ca sử dụng và lớp tham gia ca sử dụng.

Tiền và hậu điều kiện của phương thức. Sau khi áp dụng các bất biến lớp trên, chuyển đổi qua mô hình filmstrip, biểu đồ đối tượng được sinh ra sẽ có cấu trúc như trong Hình 2.9. Cấu trúc của mô hình filmstrip đã thỏa mãn với yêu cầu đề ra, tuy nhiên chưa thể hiện được sự thay đổi của các đối tượng sau mỗi lần chuyển trạng thái. Giá trị của các thuộc tính trong một đối tượng là ngẫu nhiên, quan hệ giữa các đối tượng vẫn chưa được thiết lập. Để có thể làm rõ được các giá trị này, ta cần phải xác định được tiền và hậu điều kiện của các lời gọi phương thức. Trong Filmstrip4Frsl, mỗi lời gọi phương thức tương ứng với một bước trong ca sử dụng ở FRSL. FRSL mô tả từng bước với hai đối tượng, preSnapshot thể hiện trạng thái trước khi chuyển bước

Tuy nhiên, việc sử dụng tiền và hậu điều kiện của phương thức là không đủ để có thể xác minh và xác nhận hành vi của mô hình. Dựa vào phương pháp PCDL đã được mô tả ở Mục 1.4.3, ta cần xây dựng ràng buộc khung cho Filmstrip4Frsl , chuyển đổi sang hậu điều kiện cho phương thức. Để phù hợp hơn với Filmstrip4Frsl , một số thay đổi, lưu ý cần được áp dụng trong quá trình xây dựng ràng buộc khung. Dưới đây là mô hình dạng bảng của PCDL được áp dụng cho Filmstrip4Frsl .

BuyTicket{step:String,ticket:Ticket,customer:Customer} Ticket{value:Integer,customer:Set(Customer)} Customer{money:Integer,ticket:Set(Ticket)}			
	BuyTicket::actStep_step01()	preStep_actStep_step02	BuyTicket::actStep_step02()
BuyTicket_s	C=Set{self}	P=Set{self}	C=Set{self}

us	TC=C+preStep_actStep_step02.P self=BuyTicket{'step01', U,U} U=BuyTicket_sus - TC		TC=C self=BuyTicket{'step02', U,U} U=BuyTicket_sus - TC
Ticket_sus	U=Ticket_sus		C=Set{self.ticket} TC=C self.ticket=Ticket{U,self. customer} U=Ticket_sus - TC
Customer_sus	C=Set{self.customer} TC=C+preStep_actStep_step02.P self=Customer{money@ pre-self.ticket.value,U} U=Customer_sus - TC		C=Set{self.customer} TC=C self=Customer{U,self.tic ket} U=Customer_sus - TC

Bảng 2.1 mô tả mô hình dạng bảng tổng quát của PCDL tương tự với [bảng] được trình bày ở Mục 1.5.3. So sánh với ràng buộc khung của Filmstrip4Frsl thể hiện dưới mô hình dạng bảng, ta có thể thấy được một vài thay đổi. Tương tự với PCDL, các phương thức (actStep_step01, actStep_step02) cũng được chuyển thành các cột và các lớp (BuyTicket, Ticket, Customer) là các lớp tham gia ca sử dụng cũng được chuyển thành các hàng. Tuy nhiên khi so sánh với PCDL, Filmstrip4Frsl không có các dòng BuyTicket_del, BuyTicket_add cũng như Ticket_del, Ticket_add, Customer_del, Customer_add. Đồng thời, giữa các cột phương thức actStep_step01 và actStep_step02 có thêm một cột preStep_actStep_step02. Kí tự P trong cột thể hiện những đối tượng xuất hiện trong preSnapshot trong mô tả ca sử dụng FRSL. Khi đó, tập các đối tượng thay đổi (kí hiệu bởi kí tự TC - total change) sẽ là tập hơn của các đối tượng được bị

thay đổi sau khi gọi phương thức (kí hiệu bởi kí tự C) và các đối tượng được liệt kê trong tập P của phương thức tiếp theo. Trong phương thức `actStep_step01`, tập hợp thay đổi C của lớp `BuyTicket` là có giá trị `Set{self}` (self là đối tượng thực hiện phương thức), tập S của `preStep2` là `Set{self}` nên tập TC của phương thức `step1` có giá trị bằng `Set{self}+Set{self} = Set{self}`. Khi đó, việc xác định các đối tượng không đổi U sẽ bằng tập `BuyTicket_sus` của trạng thái trước trừ đi tập các đối tượng thay đổi TC. Việc áp dụng những thay đổi trên trong quá trình xây dựng ràng buộc khung cho `Filmstrip4Frsl` là bởi các nguyên nhân được trình bày dưới đây.

Đầu tiên, ràng buộc khung của `Filmstrip4Frsl` sẽ bỏ đi các đối tượng *del* và *add* trong mô hình dạng bảng của PCDL. Trong mô hình dạng bảng của PCDL, mỗi lớp sẽ sinh ra ba dòng tương ứng với ba trạng thái của lớp đó là *del*, *add* và *sus* tương ứng với loại bỏ, thêm mới và giữ nguyên. Thay đổi này giúp giảm tải độ phức tạp cho cả việc thiết kế ràng buộc khung cho `Filmstrip4Frsl` cùng với việc tăng khả năng xử lý của công cụ tìm kiếm mô hình Model Validator khi bỏ đi các ràng buộc không quan trọng. Đồng thời, với tính chất của `Filmstrip4Frsl` việc quan tâm đến các đối tượng *del* và *add* là không cần thiết. Hướng tiếp cận của `Filmstrip4Frsl` với kỹ thuật `Filmstrip` là sử dụng `Filmstrip` nhằm sinh ra dữ liệu của một luồng chứ không phải sinh dữ liệu cho chuỗi trạng thái bất kì thỏa mãn. Vậy nên toàn bộ đối tượng được đề cập trong lớp ca sử dụng được tạo ra ngay từ đầu trong biểu đồ đối tượng của mô hình `filmstrip`, các đối tượng này sẽ không được thêm mới hay loại bỏ đi trong toàn bộ quá trình. Các đối tượng tham gia ca sử dụng được xác định như một thuộc tính của ca sử dụng trong mô hình FRSL, giúp việc liệt kê các đối tượng tham gia ca sử dụng trở nên dễ dàng hơn.

Tiếp theo, hậu điều kiện của một bước sẽ kết hợp với tiền điều kiện của bước tiếp theo để làm cơ sở xây dựng ràng buộc khung cho bước đó. Việc xây dựng ràng buộc khung bằng phương pháp PCDL chỉ quan tâm đến hậu điều kiện mà không đề cập tới tiền điều kiện, tiền điều kiện chỉ được sử dụng làm. Tuy nhiên, đối tượng `preSnapshot` trong mô hình FRSL không chỉ thể hiện điều kiện để bắt đầu một bước mà còn thể hiện hành động của người dùng với hệ thống. Để thuận tiện hơn cho việc mô tả trạng thái giữa hai bước, `Filmstrip4Frsl` sẽ gộp trạng thái sau khi kết thúc một bước và trạng thái trước khi xảy ra bước tiếp theo làm một. Khi đó, tập hợp các đối tượng bị thay đổi sau một phương thức không những chỉ nằm trong tập những đối tượng được đề cập trong hậu điều kiện của phương thức đó nữa mà gồm cả các đối tượng được liệt kê trong tiền điều kiện của phương thức tiếp theo. Vì `Filmstrip4Frsl` mô tả luồng trong ca sử dụng bao gồm các phương thức tương ứng với các bước diễn

ra theo một trình tự nhất định, vậy nên sự kết hợp ràng buộc giữa hậu điều kiện của phương thức và tiền điều kiện của phương thức tiếp theo là không đổi.

Sau khi xác định được ràng buộc khung của các phương thức, các ràng buộc này sẽ được chuyển đổi sang hậu điều kiện của phương thức theo phương pháp được đề cập ở Mục 1.4.3.3. Các đối tượng preSnapshot của mỗi bước trong FRSL sẽ được chuyển đổi sang tiền điều kiện của phương thức, còn postSnapshot sẽ được dùng để xác định ràng buộc khung của phương thức rồi sau đó xây dựng hậu điều kiện cho phương thức. Ngoài các điều kiện trên, Filmstrip4Frsl còn có thêm tiền và hậu điều kiện riêng để xác định bước hiện tại của mỗi trạng thái, thể hiện ở Hình 2.10. Các điều kiện này ràng buộc thứ tự xảy ra các phương thức của lớp ca sử dụng, từ đó thể hiện trạng thái của hệ thống theo thứ tự các bước của một luồng trong ca sử dụng.

```
context BuyTicket::actStep_step02()
  pre startStep:
    step='step01'
  post endStep:
    step='step02'
```

Hình 2.10 Ràng buộc thứ tự thực hiện của các phương thức.

Toàn bộ bất biến lớp và các ràng buộc tiền và hậu điều kiện của phương thức step1 step2 được thể hiện trong Hình 2.11 dưới đây. Trong đó, hậu điều kiện unChange thể hiện các ràng buộc với các đối tượng không thay đổi U trong mô hình cải tiến PCDL được trình bày bên trên. Các đối tượng trong preSnapshot sẽ được chuyển thành tiền điều kiện preCon, đối tượng trong postSnapshot sẽ được sinh ra tương ứng với hậu điều kiện postCon theo phương pháp chuyển đổi PCDL sang hậu điều kiện.

```
context BuyTicket::actStep_step01()
  pre startStep:
    step=''
  post endStep:
    step='step01'
  post unChange:
    Ticket.allInstances→forall(x|x@pre=x and x.value@pre=x.value and x.customer@pre=x.customer)
  post postCon:
    customer@pre=customer and customer.money@pre=customer.money+ticket.value

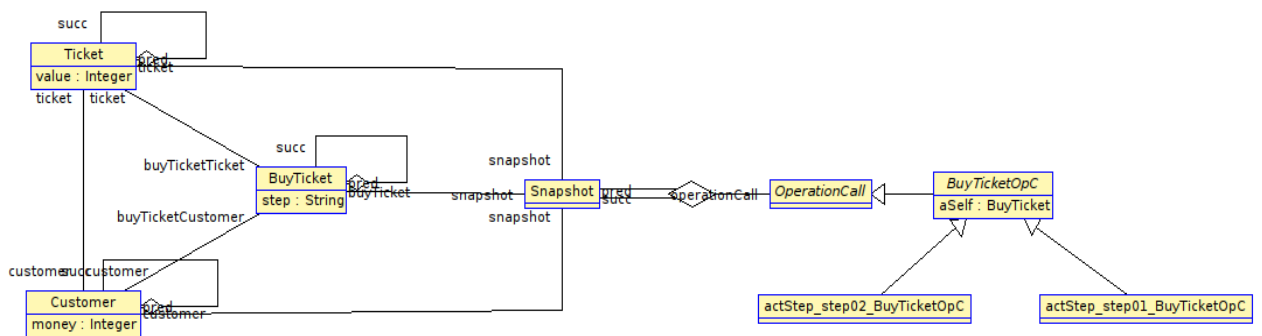
context BuyTicket::actStep_step02()
  pre startStep:
    step='step01'
  post endStep:
    step='step02'
  pre preCon:
    customer.money>0 and ticket.value>0
    and customer.money>ticket.value
  post postCon:
    customer@pre=customer and customer.money@pre=customer.money and customer.ticket=ticket
    and ticket@pre=ticket and ticket.value@pre=ticket.value
```

Hình 2.11 Tiền và hậu điều kiện hoàn chỉnh của các phương thức.

2.3.3. Biểu diễn *Filmstrip4Frsl* dưới dạng mô hình *filmstrip*

Sau khi mô tả ca sử dụng bằng *Filmstrip4Frsl* dưới dạng mô hình ứng dụng, mô hình này sẽ được sử dụng làm đầu vào của plugin *filmstrip* để chuyển tự động sang dạng biểu diễn mô hình *filmstrip*. Phần này sẽ mô tả các thành phần của *Filmstrip4Frsl* biểu diễn ở dạng mô hình *filmstrip*.

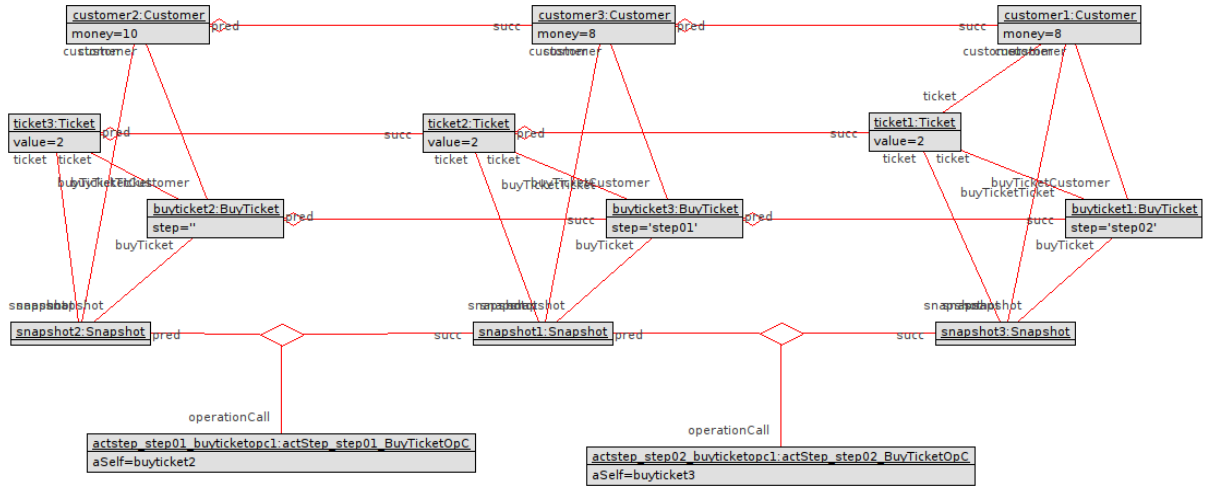
Hình 2.12 dưới đây mô tả biểu đồ lớp của các lớp trong *Filmstrip4Frsl* sau khi được chuyển qua mô hình *filmstrip*. Định nghĩa các lớp được giữ nguyên với các thuộc tính và phương thức. Ngoài các quan hệ được định nghĩa sẵn trong *Filmstrip4Frsl*, các lớp trong *Filmstrip4Frsl* còn có thêm quan hệ với tiền tố *PredSucc* nối với chính bản thân lớp đó với, thể hiện trạng thái trước và sau của một đối tượng trong hai trạng thái liên tiếp. Các lớp mới được sinh ra là các thành phần mặc định của mô hình *filmstrip* bao gồm lớp *Snapshot* và lớp *OperationCall*. Mỗi phương thức của một lớp sẽ tạo ra một lớp *OperationCall* tương ứng với hậu tố *OpC*. Như trong mô hình ở Hình 2.6, lớp *BuyTicket* có hai phương thức *actStep_step01* và *actStep_step02* thể hiện hai bước của ca sử dụng *BuyTicket*, hai phương thức này được chuyển thành hai lớp *actStep_step01_BuyTicketOpC*, *actStep_step02_BuyTicketOpC*. Hai lớp này kế thừa lớp *BuyTicketOpC*, cho thấy đây là các lớp *OperationCall* được sinh ra từ các phương thức của lớp *BuyTicket*.



Hình 2.12 Biểu diễn mô hình *Filmstrip4Frsl* dưới dạng mô hình *filmstrip*.

Hình 2.13 mô tả biểu đồ đối tượng của một luồng trong ca sử dụng có hai bước, tương ứng với ba trạng thái được theo dõi bởi ba đối tượng lần lượt là *snapshot2*, *snapshot1* và *snapshot3*. Biểu đồ đối tượng này là kết quả sau khi sinh dữ liệu từ mô hình *filmstrip*. Toàn bộ các đối tượng của trạng thái đầu tiên được thể hiện bằng các đối tượng trong khu vực nét đứt, gồm có bốn đối tượng có kiểu lớp tham gia ca sử dụng và một đối tượng lớp ca sử dụng *BuyTicket*. Trong một trạng thái, chỉ có duy nhất một đối tượng thuộc lớp ca sử dụng *BuyTicket* tượng trưng cho ca sử dụng tại

trạng thái đó. Đối tượng ca sử dụng này liên kết với các đối tượng khác trong cùng trạng thái bằng liên kết có tiền tố FRSL, cùng với thuộc tính step có giá trị là tên bước để dẫn đến trạng thái đó.



Hình 2.13 Biểu đồ đối tượng của mô hình Filmstrip4Frsl dưới dạng mô hình filmstrip sau khi đã được sinh dữ liệu.



Hình 2.14 Mô hình filmstrip được sinh ra dưới cú pháp của công cụ USE.

Các đối tượng giữa hai trạng thái liên tiếp nhau có các liên kết với vai trò pred và succ, thể hiện trạng thái của một đối tượng trong ca sử dụng trước và sau khi thực hiện một bước. Đối tượng của lớp OperationCall actstep_step01_buyticketop1 và actstep_step02_buyticketop1 được sử dụng để chỉ lời gọi phương thức dẫn đến sự

chuyển tiếp giữa ba trạng thái liên tiếp nhau qua hai bước. Tiền và hậu điều kiện được mô tả hành vi của các phương thức được chuyển thành các bất biến lớp của các lớp OperationCall.

2.4. Chuyển đổi từ mô hình FRSL sang mô hình Filmstrip4Frsl

Như đã mô tả trong Mục 2.2. Tổng quan về phương pháp, bước đầu tiên sau khi có được mô tả ca sử dụng ở dạng mô hình FRSL là thực hiện chuyển đổi mô hình đó sang mô hình Filmstrip4Frsl. Mô hình Filmstrip4Frsl sinh ra sẽ được thể hiện dưới dạng mã nguồn của công cụ USE, có định dạng là các tệp có đuôi .use nên việc chuyển đổi mô hình sẽ được thực hiện bằng phương pháp chuyển đổi mô hình sang văn bản. Công cụ được sử dụng để chuyển đổi mô hình là Acceleo đã được giới thiệu trong Mục 1.6. Phần này sẽ tập trung mô tả các luật chuyển để chuyển đổi từ mô hình FRSL sang mô hình Filmstrip4Frsl cùng với các luật để sinh ra tệp cấu hình cho quá trình sinh dữ liệu.

2.4.1. Luật chuyển đổi mô hình từ FRSL sang mô hình Filmstrip4Frsl

Đầu tiên, ta cần xác định đầu ra của quá trình chuyển đổi sang mô hình Filmstrip4Frsl, bao gồm các thành phần của mô hình Filmstrip4Frsl. Các thành phần này đã được đề cập trong Mục 2.3, gồm có: *lớp ca sử dụng*, *các lớp tham gia ca sử dụng*, *quan hệ giữa lớp ca sử dụng và các lớp tham gia ca sử dụng*, *quan hệ giữa các lớp tham gia ca sử dụng*, *bất biến lớp* ràng buộc cấu trúc của mô hình Filmstrip4Frsl và cuối cùng là *tiền và hậu điều kiện của các phương thức* thể hiện hành vi của ca sử dụng trong mỗi bước.

Luật chuyển 1. Luật chuyển thành lớp tham gia ca sử dụng.

- Mỗi đối tượng có kiểu Class thuộc package model sẽ được chuyển sang thành một *lớp tham gia ca sử dụng*.
- Tên và các thuộc tính nguyên thủy của đối tượng có kiểu Class thuộc package model sẽ được chuyển thành tên và thuộc tính của *lớp tham gia ca sử dụng*.

Luật chuyển 2. Luật chuyển thành lớp ca sử dụng.

- Một đối tượng có kiểu Class trong package usecase của mô hình FRSL sẽ được chuyển sang một *lớp ca sử dụng* trong mô hình Filmstrip4Frsl.
- Tên và các thuộc tính nguyên thủy của đối tượng có kiểu Class trong package usecase được chuyển trực tiếp sang tên và các thuộc tính của *lớp ca sử dụng* trong mô hình Filmstrip4Frsl.

- Bổ sung thêm thuộc tính *step* có kiểu String vào *lớp ca sử dụng* nhằm mô tả bước dẫn đến trạng thái hiện tại của ca sử dụng.
- Chuyển tên các Step trong mô hình FRSL thành tên các phương thức trong lớp ca sử dụng trong mô hình Filmstrip4Frsl. Vì mỗi Step có một thuộc tính *nextStep* để trỏ vào Step tiếp theo, việc lấy các Step được thực hiện qua thuật toán DFS, thể hiện ở Hình 2.15 dưới.

```
[query public getAllOperationsOfUsecase(step : Step) : OrderedSet(Step) =
  if (step.ocIsUndefined()) then
    OrderedSet{}
  else if (not step.altFlow->isEmpty()) then
    step.nextStep.getAllOperationsOfUsecase()->prepend(step.altFlow->first().altStep)->prepend(step)
  else
    step.nextStep.getAllOperationsOfUsecase()->prepend(step)
  endif
endif
/]
```

Hình 2.15 Truy vấn để lấy toàn bộ các phương thức thành một OrderedSet.

Luật chuyển 3. Luật chuyển thành quan hệ giữa lớp ca sử dụng và các lớp tham gia ca sử dụng

- Các thuộc tính có kiểu khác PrimitiveType (đây là các lớp người dùng tự định nghĩa) của đối tượng có kiểu Class trong package usecase của mô hình FRSL sẽ được chuyển thành các quan hệ có kiểu aggregation giữa lớp ca sử dụng và lớp tham gia ca sử dụng trong mô hình Filmstrip4Frsl.
- Tên của các quan hệ này được tạo thành tiền tố FRSL_ kèm với tên thuộc tính.
- Có hai thành phần tham gia quan hệ là lớp ca sử dụng và lớp tham gia ca sử dụng. Tên vai trò của lớp tham gia ca sử dụng trong quan hệ được lấy từ tên của thuộc tính trong ca sử dụng, tên vai trò của lớp ca sử dụng được xác định bằng tên ca sử dụng kết hợp với tên thuộc tính.
- Nếu thuộc tính trong FRSL là một tập hợp (Set hoặc Bag), lớp tham gia ca sử dụng tham gia quan hệ có số lượng đối tượng tham gia quan hệ là lớn hơn hoặc bằng 1 ([1..*]).
- Nếu thuộc tính trong FRSL không phải một tập hợp, lớp tham gia ca sử dụng tham gia quan hệ có số lượng đối tượng tham gia quan hệ bằng 1 ([1..1]).
- Số lượng đối tượng lớp ca sử dụng tham gia quan hệ là từ 0 đến 1 ([0..1]).

Luật chuyển 4. Luật chuyển thành quan hệ giữa các lớp tham gia ca sử dụng.

- Các đối tượng có kiểu AssociationClass trong package model sẽ được chuyển thành quan hệ giữa các lớp tham gia ca sử dụng.

- Tên, vai trò và kiểu của các thành phần tham gia quan hệ được giữ nguyên so với các thông tin của đối tượng AssociationClass.
- Trong đối tượng AssociationClass, nếu thuộc tính có số lượng phần tử lớn hơn hoặc bằng 1 (Set hoặc Bag có cận dưới bằng 1) thì trong quan hệ được sinh ra số lượng đối tượng của thành phần này tham gia liên kết có giá trị lớn hơn hoặc bằng 0 và nhỏ hơn x ($[0..x]$ với x lớn hơn hoặc bằng 1).

Luật chuyển 5. Luật chuyển thành các bất biến lớp.

```
context [usecase.name/] inv usecaseContrain:
    [usecase.name/].allInstances()->size=1
```

Hình 2.16 Khuôn mẫu để tạo bất biến lớp ràng buộc có một đối tượng lớp ca sử dụng.

- Bất biến lớp của lớp ca sử dụng được sinh ra với khuôn mẫu như Hình 2.16, thể hiện chỉ có duy nhất một đối tượng ca sử dụng trong một trạng thái.

```
context [usecase.name/] inv usecaseConnectAll:
[for (modelClass : Class | frslModel.getModelClassInUsecase(usecaseName)) before ('\\t') separator (' and\\n\\t')]
[modelClass.name/].allInstances()->forAll(x)[for (p : Property |
    usecase.getPropertyOfSpecificTypeFromUsecase(modelClass)) separator (' + ')]
x.[usecaseName.toLowerFirst()/][p.name.toUpperFirst()/]->size()[/for] = 1[/for]
```

Hình 2.17 Khuôn mẫu để tạo bất biến lớp ràng buộc quan hệ giữa lớp ca sử dụng và các lớp tham gia ca sử dụng.

- Bất biến lớp của các lớp tham gia ca sử dụng được sinh ra với khuôn mẫu thể hiện ở Hình 2.17. Ràng buộc này thể hiện với mỗi đối tượng của lớp tham gia ca sử dụng, chỉ có duy nhất một liên kết với lớp ca sử dụng.

Luật chuyển 6. Luật chuyển thành các tiền, hậu điều kiện của phương thức.

```
context [usecaseName/]:[step.getOperationName()/]()
pre beforeStep: step='[if (not step.preStep.ocIsUndefined())][step.preStep.name/][if]'
post afterStep: step='[step.name/]
```

Hình 2.18 Khuôn mẫu để sinh ràng buộc thứ tự thực hiện của phương thức.

- Với mỗi Step trong mô hình FRSL, ta sinh ra 2 ràng buộc của phương thức có tên trùng với tên Step với khuôn mẫu như Hình 2.18. Hai ràng buộc này có tác dụng ràng buộc các lời gọi phương thức theo một thứ tự đã xác định trước. Theo đó, ràng buộc beforeStep yêu cầu thuộc tính *step* của *lớp ca sử dụng* phải bằng giá trị nhất định mới có thể thực hiện phương thức này.
- Với mỗi đối tượng được mô tả trong preSnapshot của FRSL sẽ được chuyển thành các tiền điều kiện trong phương thức tương ứng.

2.4.2. Luật chuyển đổi tệp cấu hình cho Model Validator

Như đã đề cập trong Mục 2.2, để có thể sinh dữ liệu tự động cho mô hình Filmstri4Frsl bằng plugin Model Validator cần một tệp cấu hình có đuôi .properties. Tệp cấu hình này sẽ xác định không gian giá trị của các thuộc tính của đối tượng, số lượng liên kết cũng như số lượng đối tượng. Cấu hình của một đối tượng được mô tả trong tệp cấu hình có định dạng như trong Hình 2.19. Các lớp, thuộc tính, liên kết được thêm hậu tố _min và _max thể hiện số lượng đối tượng thấp nhất và cao nhất mà đối tượng đó có thể đạt được. Các giá trị bằng -1 ở các trường thể hiện số lượng sẽ tương ứng với vô hạn. Hình 2.19 còn thể hiện số lượng đối tượng thuộc lớp ca sử dụng BuyTicket có giá trị nhỏ và lớn nhất tương ứng là 0 và 10. Các thuộc tính của lớp được thể hiện ở dạng *tên lớp_tên thuộc tính*. Trường số lượng thuộc tính step đều có giá trị lớn nhất là -1 thể hiện các thuộc tính này bắt buộc có giá trị với mọi đối tượng của lớp BuyTicket.

```
# ----- BuyTicket
BuyTicket_min = 3
BuyTicket_max = 3

BuyTicket_step = Set{'', 'step01', 'step02'}
BuyTicket_step_min = -1
BuyTicket_step_max = -1
```

Hình 2.19 Định dạng cấu hình của một đối tượng.

Để có thể sinh tự động tệp cấu hình này, ta cần xác định các thành phần của một tệp cấu hình. Sử dụng Model Validator cho mô hình Filmstrip4Frsl yêu cầu tệp cấu hình phải lưu các thông tin của: *đối tượng của lớp ca sử dụng, đối tượng của lớp tham gia ca sử dụng, đối tượng của lớp Snapshot, liên kết giữa các lớp tham gia ca sử dụng, liên kết giữa các lớp tham gia ca sử dụng và ca sử dụng và cuối cùng là liên kết giữa của lớp Snapshot và các lớp ca sử dụng/lớp thuộc ca sử dụng*. Đầu vào để lấy được các thông tin này sẽ dựa vào mô hình FRSL với các luật chuyển sẽ được trình bày dưới đây.

Luật chuyển 1. Luật chuyển thành cấu hình đối tượng lớp ca sử dụng

- Số lượng đối tượng lớp ca sử dụng có giá trị thấp nhất là số bước ít nhất để thực hiện ca sử dụng, giá trị cao nhất là số bước lớn nhất mà ca sử dụng có thể đạt được (như đã mô tả ở Mục 2.3.1, mỗi trạng thái chỉ có duy nhất một đối tượng lớp ca sử dụng).

- Số lượng các thuộc tính trong lớp ca sử dụng đều được để giá trị mặc định là -1 (tương ứng với điều kiện tất cả các thuộc tính của đối tượng đều phải có giá trị). Điều này là bởi Model Validator sẽ gặp lỗi trong quá trình giải bài toán có ràng buộc ở thuộc tính có giá trị là rỗng. Như trong Hình 2.19, giá trị số lượng thuộc tính step của lớp BuyTicket có hai cận lớn và nhỏ nhất đều được đặt bằng -1.
- Giá trị của thuộc tính *step* là tập tên các bước trong một ca sử dụng.

Luật chuyển 2. Luật chuyển thành cấu hình các đối tượng của lớp tham gia ca sử dụng.

- Số lượng các đối tượng của lớp tham gia ca sử dụng có các giá trị lớn nhất, nhỏ nhất bằng n lần số bước nhỏ nhất, lớn nhất mà ca sử dụng có thể đạt được, với n là số thuộc tính của lớp tham gia ca sử dụng trong ca sử dụng được xét. Ví dụ như trong ca sử dụng BuyTicket, có một thuộc tính có kiểu Ticket. Vậy nên số lượng các đối tượng của lớp Ticket sẽ bằng số lượng các đối tượng lớp BuyTicket.
- Tương tự như lớp ca sử dụng, số lượng thuộc tính của các đối tượng của lớp tham gia ca sử dụng đều có giá trị mặc định bằng -1.

Luật chuyển 3. Luật chuyển thành cấu hình các đối tượng của Snapshot

- Số lượng đối tượng của lớp Snapshot bằng với số lượng lớp ca sử dụng - tức là được xác định trong khoảng số bước nhỏ nhất và lớn nhất của ca sử dụng.

Luật chuyển 4. Luật chuyển thành cấu hình các liên kết giữa các lớp tham gia ca sử dụng.

- Đối với các liên kết được định nghĩa trong mô hình FRSL của ca sử dụng, số lượng các liên kết không bị giới hạn, có số lượng nhỏ nhất là 0, lớn nhất là vô cùng (thể hiện bởi giá trị -1)
- Đối với các liên kết PredSucc được sinh ra trong quá trình chuyển đổi sang mô hình filmstrip, số lượng các liên kết này bằng số lượng các đối tượng của lớp tham gia liên kết trừ đi 1. Từ đó việc xác định khoảng số lượng liên kết này được xác định dựa theo việc xác định số lượng đối tượng ở luật chuyển 1 và 2.

Luật chuyển 5. Luật chuyển thành cấu hình các liên kết giữa các lớp tham gia ca sử dụng và lớp ca sử dụng.

- Số lượng các liên kết giữa các lớp tham gia ca sử dụng và lớp ca sử dụng bằng số đối tượng thuộc các lớp tham gia ca sử dụng, bởi vì mỗi đối tượng thuộc các

lớp tham gia ca sử dụng phải có một và chỉ một liên kết này. Từ đó việc xác định khoảng số lượng liên kết này được xác định dựa theo việc xác định số lượng đối tượng ở luật chuyển 2.

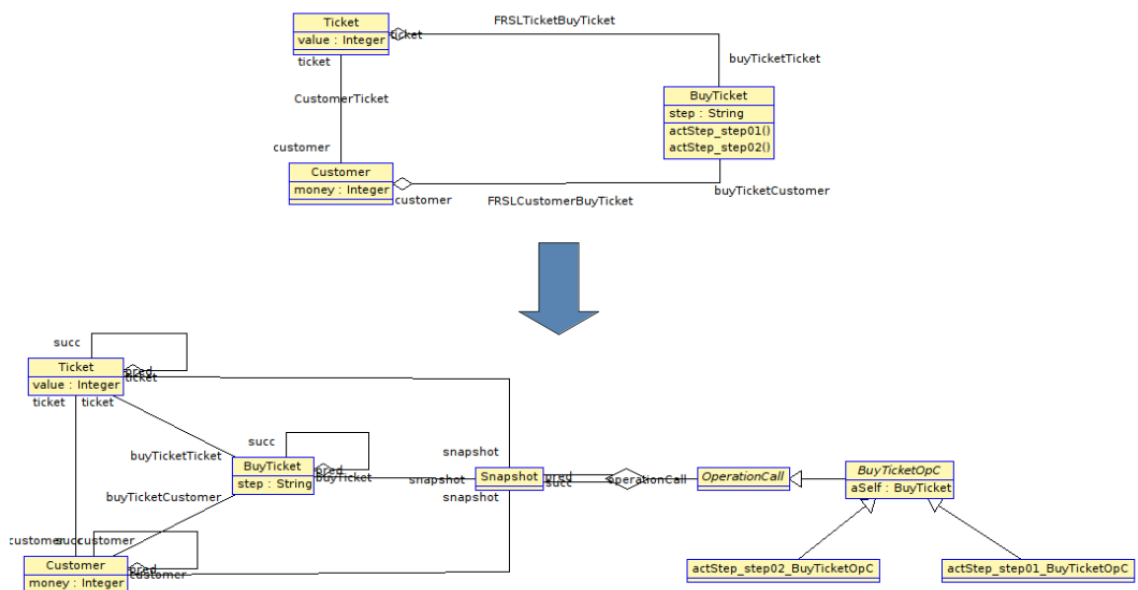
Luật chuyển 6. Luật chuyển thành cấu hình liên kết giữa của lớp Snapshot và các lớp ca sử dụng/lớp thuộc ca sử dụng.

- Số lượng liên kết giữa lớp Snapshot và các lớp ca sử dụng/lớp thuộc ca sử dụng bằng số đối tượng thuộc lớp ca sử dụng/lớp thuộc ca sử dụng tương ứng bởi mỗi đối tượng của một lớp đều phải có một và chỉ một liên kết với lớp Snapshot. Từ đó việc xác định khoảng số lượng liên kết này được xác định dựa theo việc xác định số lượng đối tượng ở luật chuyển 1 và 2.

2.5. Sinh dữ liệu ca kiểm thử từ mô hình Filmstrip4Frsl

Sau khi thu được mô hình Filmstrip4Frsl và tệp cấu hình sinh dữ liệu từ các luật chuyển được xác định ở Mục 2.4 bằng công cụ Acceleo, việc sinh dữ liệu ca kiểm thử sẽ được thực hiện tự động bằng các plugin của công cụ USE. Phần này sẽ tập trung vào quá trình sử dụng các plugin Filmstrip và Model Validator để sinh dữ liệu cho mô hình Filmstrip4Frsl.

Đầu tiên, người dùng cần sử dụng plugin chuyển mô hình Filmstrip4Frsl ở dạng mô hình hình ứng dụng sang mô hình Filmstrip4Frsl ở dạng mô hình filmstrip. Quá trình chuyển đổi được thể hiện trong Hình 2.20.



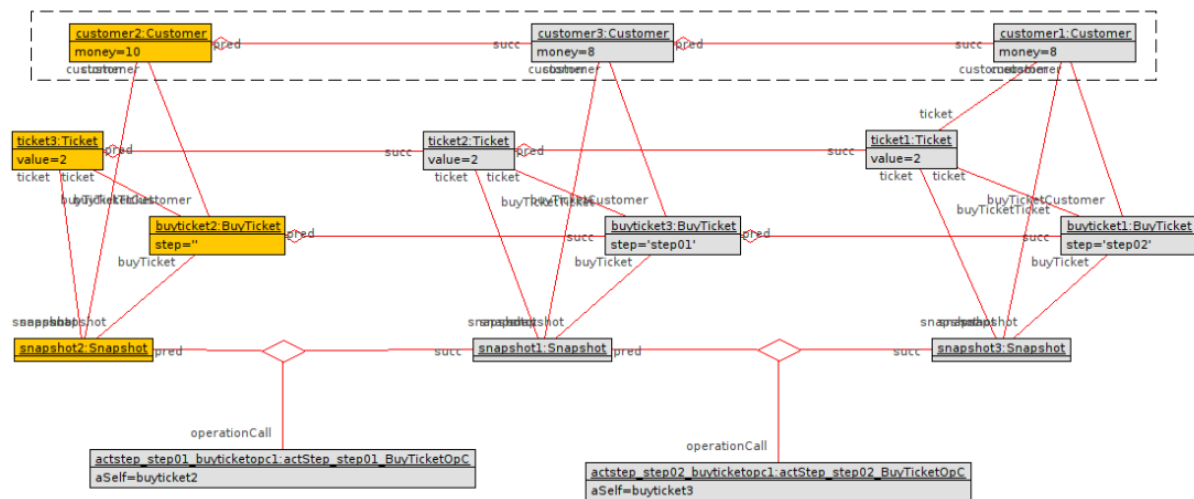
Hình 2.20 Sự chuyển đổi của mô hình Filmstrip4Frsl từ dạng mô hình ứng dụng sang dạng mô hình filmstrip.

Chi tiết về mô hình Filmstrip4Frsl ở dạng mô hình filmstrip đã được trình bày cụ thể trong Mục 2.3.2. Biểu diễn Filmstrip4Frsl dưới dạng mô hình filmstrip. Tiếp theo, Model Validator yêu cầu đầu vào gồm một mô hình để sinh dữ liệu và tệp cấu hình sinh dữ liệu. Trong phương pháp được đề cập ở khóa luận này, mô hình được làm đầu vào cho Model Validator chính là mô hình Filmstrip4Frsl ở dạng mô hình filmstrip đã được chuyển đổi ở trên. Từ đây, người dùng đã có thể trực tiếp sử dụng Model Validator cho mô hình filmstrip thu được từ quá trình chuyển đổi để tìm kiếm mô hình và sinh dữ liệu. Tuy nhiên khi đó khoảng tìm kiếm cho các đối tượng trong mô hình đều là mặc định, người dùng cần chỉnh thủ công các giá trị cấu hình để có thể sử dụng Model Validator. Quá trình này rất phức tạp và đòi hỏi người sử dụng hiểu rõ cả plugin Model Validator và phương pháp Filmstrip4Frsl, đồng thời việc sửa đổi mô hình gốc FRSL sẽ khiến việc cấu hình phải thực hiện lại từ đầu. Do đó, tệp cấu hình cho Model Validator sẽ được sinh tự động với các luật chuyển được đề cập trong Mục 2.4.2 và có định dạng như Hình 2.21 dưới đây.

<pre> # ----- actStep_step02_BuyTicketOpC actStep_step02_BuyTicketOpC_min = 1 actStep_step02_BuyTicketOpC_max = 1 # ----- actStep_step01_BuyTicketOpC actStep_step01_BuyTicketOpC_min = 1 actStep_step01_BuyTicketOpC_max = 1 # ----- BuyTicketOpC BuyTicketOpC_aSelf_min = -1 BuyTicketOpC_aSelf_max = -1 # ----- Snapshot Snapshot_min = 3 Snapshot_max = 3 # Filmstrip (pred:Snapshot, succ:Snapshot, operationCall:OperationCall) ----- Filmstrip_min = 2 Filmstrip_max = 2 # ----- OperationCall # ----- Customer Customer_min = 3 Customer_max = 3 Customer_money_min = -1 Customer_money_max = -1 # PredSuccCustomer (pred:Customer, succ:Customer) ----- PredSuccCustomer_min = 2 PredSuccCustomer_max = 2 # SnapshotCustomer (customer:Customer, snapshot:Snapshot) ----- SnapshotCustomer_min = 3 SnapshotCustomer_max = 3 # CustomerTicket (customer:Customer, ticket:Ticket) ----- CustomerTicket_min = 1 CustomerTicket_max = 1 # FRSLCustomerBuyTicket (customer:Customer, buyTicketCustomer:BuyTicket) ----- FRSLCustomerBuyTicket_min = 3 FRSLCustomerBuyTicket_max = 3 </pre>	<pre> # ----- Ticket Ticket_min = 3 Ticket_max = 3 Ticket_value_min = -1 Ticket_value_max = -1 # SnapshotTicket (ticket:Ticket, snapshot:Snapshot) ----- SnapshotTicket_min = 3 SnapshotTicket_max = 3 # PredSuccTicket (pred:Ticket, succ:Ticket) ----- PredSuccTicket_min = 2 PredSuccTicket_max = 2 # FRSLTicketBuyTicket (ticket:Ticket, buyTicketTicket:BuyTicket) ----- FRSLTicketBuyTicket_min = 3 FRSLTicketBuyTicket_max = 3 # ----- BuyTicket BuyTicket_min = 3 BuyTicket_max = 3 BuyTicket_step = Set{'step01', 'step02'} BuyTicket_step_min = -1 BuyTicket_step_max = -1 # PredSuccBuyTicket (pred:BuyTicket, succ:BuyTicket) ----- PredSuccBuyTicket_min = 2 PredSuccBuyTicket_max = 2 # SnapshotBuyTicket (buyTicket:BuyTicket, snapshot:Snapshot) ----- SnapshotBuyTicket_min = 3 SnapshotBuyTicket_max = 3 # ----- aggregationcyclefreeness = on forbiddensharing = on </pre>
--	--

Hình 2.21 Tệp cấu hình sinh dữ liệu đầy đủ.

Khi đã có đầy đủ các tạo tác để sử dụng Model Validator, kết quả của việc thực thi sẽ được thể hiện như Hình 2.22. Toàn bộ các đối tượng được sinh ra đều thỏa mãn các cấu hình được thiết lập cùng các ràng buộc của mô hình Filmstrip4Frsl. Từ biểu đồ đối tượng thu được này, ta có thể lấy được thông tin của các đối tượng trong một trạng thái của hệ thống tại một thời điểm trong ca sử dụng.



Hình 2.22 Biểu đồ đối tượng được sinh ra sau khi sử dụng plugin Model Validator

Các đối tượng được đôi đậm bằng màu cam thể hiện là các đối tượng trong cùng một trạng thái. Trạng thái này tương ứng với một Snapshot theo định nghĩa của FRSL, các đối tượng trong một trạng sẽ được quản lý bởi một đối tượng snapshot trong mô hình filmstrip. Các trạng thái liên tiếp của cùng một đối tượng thuộc lớp Customer được thể hiện trong hình vuông có nét đứt, giữa các đối tượng có liên kết với vai trò pred và succ tương ứng với trạng thái trước và sau. Giá trị và liên kết giữa các đối tượng được giữ nguyên hoặc thay đổi tùy theo mô tả của phương thức, từ đó theo dõi được hành vi của các đối tượng trong ca sử dụng quá trình thực hiện chuyển bước.

Quá trình thực hiện sinh dữ liệu của Model Validator có thể được biểu diễn bằng tập hợp các lệnh trong công cụ USE thể hiện ở Hình 2.23. Từ các lệnh này, ta có thể tái hiện lại quá trình sinh dữ liệu, đồng thời có thể được sử dụng để bóc tách các giá trị của các đối tượng để sử dụng cho các mục đích khác.

```

!new actStep_step02_BuyTicketOpC('actstep_step02_buyticketopc1')
!new actStep_step01_BuyTicketOpC('actstep_step01_buyticketopc1')
!new Snapshot('snapshot1')
!new Snapshot('snapshot2')
!new Snapshot('snapshot3')
!new Customer('customer1')
!new Customer('customer2')
!new Customer('customer3')
!new Ticket('ticket1')
!new Ticket('ticket2')
!new Ticket('ticket3')
!new BuyTicket('buyticket1')
!new BuyTicket('buyticket2')
!new BuyTicket('buyticket3')
!customer1.money := 8
!customer2.money := 10
!customer3.money := 8
!insert (customer1,buyticket1) into FRSLCustomerBuyTicket
!insert (customer2,buyticket2) into FRSLCustomerBuyTicket
!insert (customer3,buyticket3) into FRSLCustomerBuyTicket
!insert (customer1,snapshot3) into SnapshotCustomer
!insert (customer2,snapshot2) into SnapshotCustomer
!insert (customer3,snapshot1) into SnapshotCustomer
!insert (ticket2,ticket1) into PredSuccTicket
!insert (ticket3,ticket2) into PredSuccTicket
!insert (buyticket2,buyticket3) into PredSuccBuyTicket
!insert (buyticket3,buyticket1) into PredSuccBuyTicket
!insert (customer1,ticket1) into CustomerTicket
!ticket1.value := 2
!ticket2.value := 2
!ticket3.value := 2
!actstep_step02_buyticketopc1.aSelf := buyticket3
!actstep_step01_buyticketopc1.aSelf := buyticket2
!insert (customer2,customer3) into PredSuccCustomer
!insert (customer3,customer1) into PredSuccCustomer
!insert (ticket1,snapshot3) into SnapshotTicket
!insert (ticket2,snapshot1) into SnapshotTicket
!insert (ticket3,snapshot2) into SnapshotTicket
!insert (snapshot1,snapshot3,actstep_step02_buyticketopc1) into Filmstrip
!insert (snapshot2,snapshot1,actstep_step01_buyticketopc1) into Filmstrip
!buyticket1.step := 'step02'
!buyticket2.step := ''
!buyticket3.step := 'step01'
!insert (buyticket1,snapshot3) into SnapshotBuyTicket
!insert (buyticket2,snapshot2) into SnapshotBuyTicket
!insert (buyticket3,snapshot1) into SnapshotBuyTicket
!insert (ticket1,buyticket1) into FRSLTicketBuyTicket
!insert (ticket2,buyticket3) into FRSLTicketBuyTicket
!insert (ticket3,buyticket2) into FRSLTicketBuyTicket

```

Hình 2.23 Các câu lệnh mô tả quá trình sinh dữ liệu của plugin Model Validator

2.6. Tổng kết chương

Chương này đã trình bày quy trình cụ thể của phương pháp sinh dữ liệu kiểm thử từ đặc tả ca sử dụng, các bước cùng thành phần trong quá trình thực hiện. Khóa luận cũng đã đề xuất phương pháp cải tiến cách xây dựng ràng buộc khung cho phù hợp với vấn đề cần giải quyết. Ngoài ra, chương này cũng giới thiệu mô hình Filmstrip4Frsl, cách sử dụng cũng như ứng dụng các công cụ chuyển đổi mô hình Acceleo và plugin thẩm định hành vi mô hình filmstrip vào phương pháp được sử dụng để sinh dữ liệu cho ca sử dụng được mô tả dưới dạng mô hình FRSL.

CHƯƠNG 3. CÀI ĐẶT VÀ THỰC NGHIỆM

Trong chương này, phương pháp đã được trình bày sẽ được cài đặt và tiến hành thử nghiệm để đánh giá kết quả.

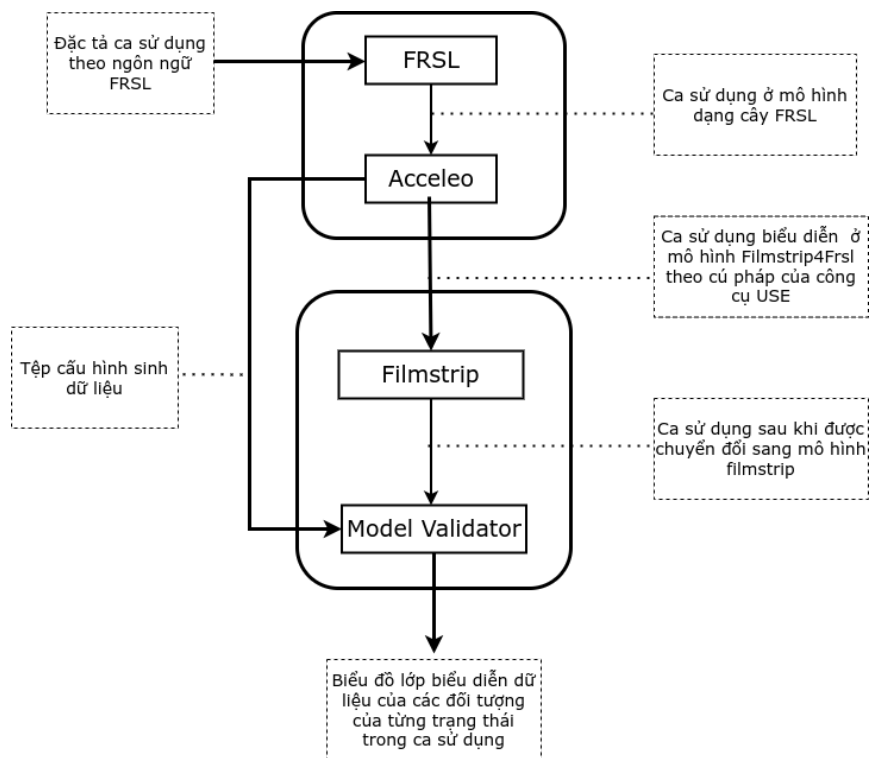
3.1. Giới thiệu

Với các kiến thức nền tảng, phương pháp và bài toán áp dụng, ta cần phát triển một công cụ giúp chuyển đổi mô hình ca sử dụng được mô tả bằng mô hình FRSL sang mô hình Filmstrip4Frsl đã được mô tả trong chương 3 làm mô hình đầu vào để tự động sinh ra dữ liệu kiểm thử.

Phần 3.2 sẽ trình bày về các thành phần và chức năng của công cụ hỗ trợ. Phần 3.3. trình bày về bài toán và kết quả thực nghiệm và cuối cùng, đánh giá công cụ sẽ được trình bày trong phần 3.4.

3.2. Công cụ hỗ trợ

Khóa luận mô tả kỹ thuật sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng từ mô hình FRSL bằng các công cụ phát triển phần mềm hướng mô hình. Phương pháp này sử dụng nhiều công cụ khác nhau được thể hiện bởi các thành phần được mô tả trong Hình 3.1.



Hình 3.1 Quy trình và các công cụ được sử dụng để sinh dữ liệu kiểm thử từ đặc tả ca sử dụng.

Trong đó, Filmstrip4Frsl Tool là công cụ được sử dụng để chuyển đổi mô hình FRSL sang mô hình Filmstrip4Frsl. Công cụ này được phát triển trên nền tảng Eclipse, đã được tích hợp với công cụ đặc tả ca sử dụng ở dưới dạng mô hình FRSL và sử dụng công nghệ Acceleo để chuyển đổi mô hình. Công cụ có hai thành phần, một thành phần được sử dụng để mô hình hóa đặc tả ca sử dụng ở dạng văn bản sang dạng mô hình FRSL, thành phần còn lại sẽ chuyển đổi mô hình FRSL sang mô hình Filmstrip4Frsl. Đầu vào của công cụ là một văn bản mô tả ca sử dụng, với cú pháp tuân theo cú pháp được định nghĩa của FRSL. Văn bản này sẽ được chuyển sang dạng mô hình FRSL qua thành phần thứ nhất của công cụ, rồi sẽ được sử dụng làm đầu vào của thành phần thứ hai để chuyển sang mô hình Filmstrip4Frsl cùng tệp cấu hình sinh dữ liệu bằng công nghệ Acceleo.

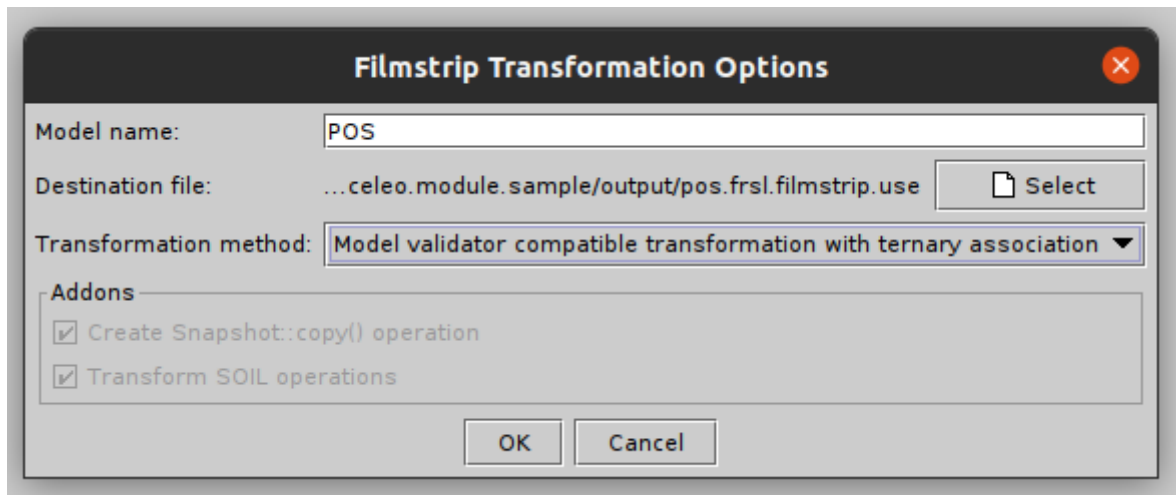


Hình 3.2 Ca sử dụng được mô tả theo cú pháp của ngôn ngữ FRSL (bên trái) và được biểu diễn theo cấu trúc cây (bên phải).

<pre> class HandleCashPayment attributes _sale: Sale sale: Sale cashPayment: CashPayment _cashPayment: CashPayment amountTendered: Real step: String operations actStep_step01() actStep_step02() actStep_step03() actStep_step04() actStep_step05() end class Sale attributes id: String total: Integer isComplete: Boolean end class Payment attributes amount: Real isDefined: Boolean end class CreditCard attributes number: String expireDate: String isDefined: Boolean end class Cashier attributes name: String isDefined: Boolean end </pre>	<pre> association ContainedIn between Sale[1] role sale SalesLineItem[1..*] role salesLineItem end association ProvidedWith between Store[0..*] role store AuthService[0..*] role authService end association UsedBy between ProductCatalog[1] role prdtCtlg Store[0..*] role store end constraints context HandleCashPayment inv usecaseContraint: Bag{ _sale, sale, cashPayment, _cashPayment } → isUnique(p p) and Bag{ _sale, sale, cashPayment, _cashPayment } → forAll(p p.isDefined) and Bag{ _sale, sale, cashPayment, _cashPayment } → size=Sale.allInstances() → size + CashPayment.allInstances() → size and HandleCashPayment.allInstances() → size=1 context HandleCashPayment inv cst1: Bag{ _sale, sale, cashPayment, _cashPayment } → forAll(p p.isDefined) and Bag{ _sale, sale, cashPayment, _cashPayment } → forAll(p p > null) Set{ _sale, sale, cashPayment, _cashPayment } → size=4 not (_sale.isUndefined or sale.isUndefined or cashPayment.isUndefined or _cashPayment. isUndefined) context HandleCashPayment inv cst2: Sale.allInstances() → size=2 and CashPayment.allInstances() → size=2 context HandleCashPayment inv cst3: HandleCashPayment.allInstances() → size=1 context HandleCashPayment::actStep_step01() pre startStep: step='1' post endStep: step='2' post test1: sale.total=7 post unchange: Bag{ _sale, sale, cashPayment, _cashPayment } → forAll(p p@pre=p) and _sale@pre=_sale and sale@pre=sale and cashPayment@pre=cashPayment and _cashPayment@pre=_cashPayment and Sale.allInstances@pre=Sale.allInstances and Payment.allInstances@pre=Payment.allInstances and CreditCard.allInstances@pre=CreditCard.allInstances and Cashier.allInstances@pre=Cashier.allInstances and AuthService.allInstances@pre=AuthService.allInstances and Date.allInstances@pre=Date.allInstances and </pre>
--	--

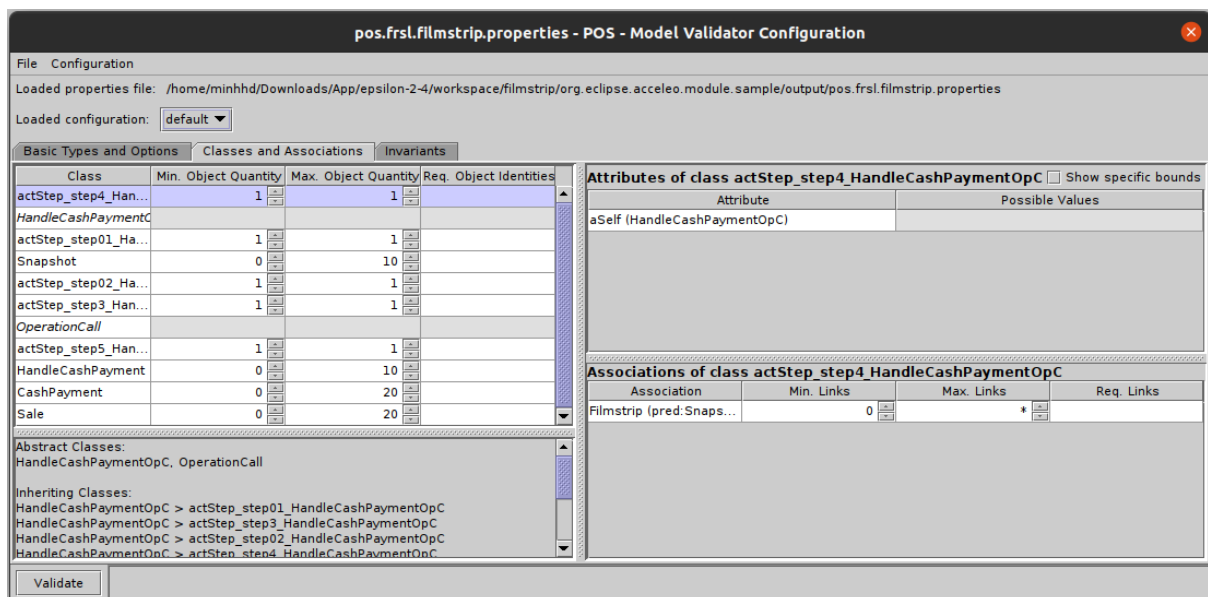
Hình 3.3 Mô hình Filmstrip4Frsl được sinh ra theo cú pháp của công cụ USE.

USE Tool là công cụ mã nguồn mở được sử dụng để chuyển mô hình Filmstrip4Frsl ở dạng mô hình ứng dụng sang mô hình Filmstrip4Frsl ở dạng mô hình filmstrip rồi sinh các giá trị cho mô hình đó. Mô hình Filmstrip4Frsl ở dạng mô hình ứng dụng thu được từ Filmstrip4Frsl Tool cần được người dùng kiểm tra và chỉnh sửa các ràng buộc tiền và hậu điều kiện của các phương thức nếu cần thiết, trước khi được chuyển đổi sang dạng mô hình filmstrip qua plugin filmstrip.



Hình 3.4 Giao diện của plugin filmstrip.

Khi đã có được mô hình dạng filmstrip của Filmstrip4Frsl, sử dụng mô hình này cùng tệp cấu hình thu được từ công cụ Filmstrip4Frsl Tool để sinh dữ liệu cho ca sử dụng bằng plugin Model Validator. Tại đây, người dùng có thể chỉnh sửa lại cấu hình sinh dữ liệu để có thể sinh được dữ liệu với các giá trị mong muốn. Đầu ra của plugin này là biểu đồ đối tượng của Filmstrip4Frsl biểu diễn ở dạng mô hình filmstrip, ta có thể sử dụng các thông tin của biểu đồ này làm dữ liệu kiểm thử cho ca sử dụng



Hình 3.5 Giao diện của plugin Model Validator.

3.3. Bài toán thực nghiệm

Ca sử dụng được lựa chọn để sử dụng cho bài toán thực nghiệm là ca sử dụng thanh toán tiền mặt được mô tả trong tài liệu “Applying UML and Patterns: An

Introduction to Object-Oriented Analysis and Design and Iterative Development” [10].
 Văn bản mô tả ca sử dụng với cú pháp FRSL được biểu diễn trong Hình 3.6.

```

usecase HandleCashPayment
  description = 'This use-case allows Customer to pay the sale by a cash payment.'
  primaryActor = Cashier

  ucPrecondition
    description = 'Customer is ready to pay the sale by a cash payment.'
    _sale: Sale;
    sale: Sale;
    (_sale, sale): _Tracks;
    [sale.isComplete = false]
  end

  ucPostcondition
    description = 'The sale is paid by a cash payment.'
    cashPayment: CashPayment;
    (sale, cashPayment): PaidBy;
    [cashPayment.amountTendered >= sale.total]
    [cashPayment.amount = sale.total]
  end

  actStep step01
    description = '1. Customer wants to pay the sale by cash.'
    from
    to
      _cashPayment: CashPayment;
    end

  actStep step02
    description = '2. Cashier enters the cash amount tendered.'
    from
      _cashPayment: CashPayment;
    to
      [ _cashPayment.amountTendered = amountTendered ]
    actions
      Cashier -> amountTendered: Real;
    end

  sysStep step3
    description = '3. System presents the balance due, and releases the cash drawer.'
    from
      _cashPayment: CashPayment;
      sale: Sale;
    to
    actions
      Cashier <- balanceDue: Real = _cashPayment.amountTendered - sale.total;
    end

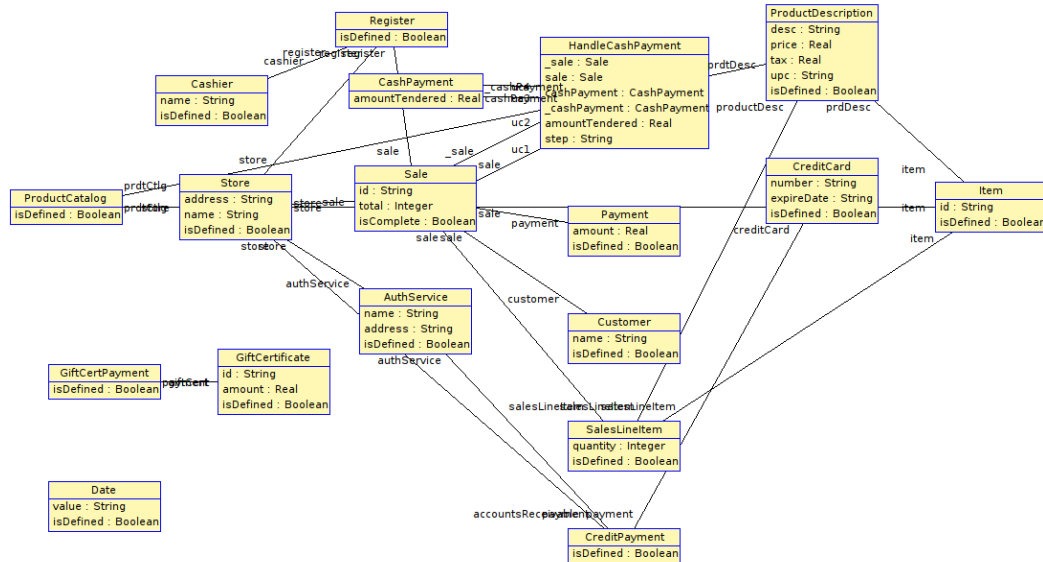
  actStep step4
    description = '4. Cashier deposits cash tendered and returns balance in cash to Customer.'
    from
      _sale: Sale;
      _cashPayment: CashPayment;
    to
      (_sale, _cashPayment): PaidBy;
    end

  sysStep step5
    description = '5. System records the cash payment.'
    from
      _cashPayment: CashPayment;
      sale: Sale;
    to
      cashPayment: CashPayment;
      (sale, cashPayment): PaidBy;
      [cashPayment.amountTendered = _cashPayment.amountTendered]
    end
end

```

Hình 3.6 Ca sử dụng HandleCashPayment được mô tả bằng ngôn ngữ FRSL.

Sau đó, mô hình FRSL này sẽ được chuyển sang thành mô hình Filmstrip4Frsl ở dạng mô hình ứng dụng và tệp cấu hình để sinh dữ liệu. Tại bước này, người dùng cần kiểm tra và xây dựng các ràng buộc khung của các phương thức trong mô hình Filmstrip4Frsl để đảm bảo hành vi của các phương thức là đúng như mong muốn.



Hình 3.7 Biểu đồ lớp mô hình Filmstrip4Frsl cho ca sử dụng HandleCashPayment.



Hình 3.8 Tệp cấu hình sinh dữ liệu cho ca sử dụng HandleCashPayment.

```

constraints
context HandleCashPayment inv usecaseContrain:
    HandleCashPayment.allInstances()→size=1

context HandleCashPayment inv usecaseConnectAll:
    Sale.allInstances→forAll(x|x.handleCashPaymentSale→size() + x.handleCashPayment_sale→size() =
    1) and
    CashPayment.allInstances→forAll(x|x.handleCashPaymentCashPayment→size() + x.
    handleCashPayment_cashPayment→size() = 1)

context HandleCashPayment::actStep_step01()
    pre beforeStep: step=''
    post afterStep: step='step01'
    post unChange:
        Sale.allInstances→forAll(x|x@pre=x and x.id@pre=x.id and x.total@pre=x.total and x.
        isComplete@pre=x.isComplete) and
        CashPayment.allInstances→forAll(x|x@pre=x and x.amountTendered@pre=x.amountTendered)

    pre preUC:
        _cashPayment.amountTendered=0 and cashPayment.amountTendered=0 and amountTendered=0
    pre preCon:
        sale.isComplete=false

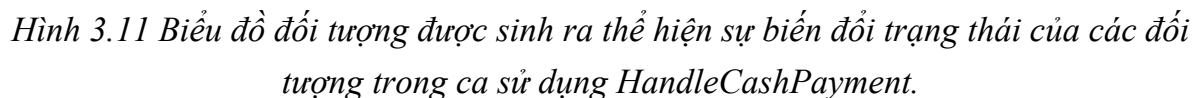
context HandleCashPayment::actStep_step02()
    pre beforeStep: step='step01'
    post afterStep: step='step02'
    post unChange:
        Sale.allInstances→forAll(x|x@pre=x and x.id@pre=x.id and x.total@pre=x.total and x.
        isComplete@pre=x.isComplete) and
        ( CashPayment.allInstances-Set{_cashPayment} )→forAll(x|x@pre=x and x.amountTendered@pre=x.
        amountTendered)
        and amountTendered@pre=amountTendered

    pre preCon:
        amountTendered<0
    post postCon:
        _cashPayment@pre=_cashPayment and _cashPayment.amountTendered=amountTendered

```

Hình 3.9 Các ràng buộc của mô hình Filmstrip4Frsl.

Cuối cùng, mô hình Filmstrip4Frsl ở dạng mô hình ứng dụng sẽ được chuyển sang dạng mô hình filmstrip qua plugin filmstrip. Mô hình filmstrip cùng tệp cấu hình sẽ làm đầu vào của plugin Model Validator để sinh dữ liệu, biểu diễn dưới dạng biểu đồ đối tượng. Người dùng có thể thay đổi một số giá trị của cấu hình Model Validator trong quá trình sử dụng plugin này, giúp sinh ra các luồng ca sử dụng hoặc các trường hợp kiểm thử mong muốn.



Phương pháp được mô tả trong khóa luận có thể thực hiện sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng, các dữ liệu được sinh ra hoàn toàn thỏa mãn các hành vi của ca sử dụng được người dùng xác định trước. Người dùng có thể sinh dữ liệu theo các luồng khác nhau của ca sử dụng, giúp kiểm thử được nhiều hành vi, trạng thái mà

hệ thống có thể đạt được trong quá trình thực hiện ca sử dụng. Ngoài ra, phương pháp trên còn có thể giúp người dùng đánh giá và xác nhận mô hình ca sử dụng được mô tả trong FRSL, xem xét tính khả thi của mô hình với các ràng buộc trong mô hình. Việc có thể sinh dữ liệu kiểm thử ngay từ quá trình đặc tả ca sử dụng có thể giúp người dùng tiết kiệm rất nhiều thời gian trong quá trình thiết kế hệ thống, đặc biệt khi các lỗi được phát hiện ngay từ giai đoạn thiết kế ca sử dụng. Kết quả của quá trình sinh dữ liệu kiểm thử này còn có thể được sử dụng để xây dựng các ca kiểm thử của kiểm thử chấp nhận, giúp tự động hóa và giảm bớt công sức trong quá trình bảo trì và phát triển hệ thống.

Tuy nhiên, phương pháp này cũng có một số hạn chế nhất định. Quá trình sinh dữ liệu bao gồm nhiều bước chuyển đổi, sử dụng nhiều công cụ khác nhau và cần sự tham gia của người dùng trong quá trình chuyển đổi. Mặc dù điều này có thể giúp người dùng dễ dàng theo dõi các tạo tác, dễ dàng chỉnh sửa để mô tả được đúng nhất hành vi của hệ thống nhưng lại gây ra sự phức tạp trong quá trình xây dựng, sửa đổi mô hình gốc. Ngoài ra, số lượng đối tượng của một ca sử dụng là rất lớn, việc sinh dữ liệu sẽ tốn rất nhiều thời gian và có độ phức tạp cao với khoảng tìm kiếm rộng. Dù vậy, phương pháp sinh tự động dữ liệu từ đặc tả ca sử dụng đã mở ra một hướng đi đầy hứa hẹn cho việc xây dựng hệ thống ở các mức trừu tượng cao hơn.

KẾT LUẬN

Khóa luận đã trình bày cụ thể kỹ thuật sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng dựa vào các công cụ được phát triển theo kỹ nghệ hướng mô hình. Quá trình sinh tự động dữ liệu từ đặc tả ca kiểm thử được thực hiện bởi sự kết hợp của nhiều công cụ, phương pháp như công cụ USE, các plugin filmstrip, Model Validator, công nghệ chuyển đổi mô hình Acceleo, kỹ thuật tìm kiếm ràng buộc khung cũng như công cụ đặc tả ca sử dụng FRSL. Ngoài ra khóa luận giới thiệu mô hình Filmstrip4Frsl, một mô hình trung gian được sử dụng để làm trung gian trong quá trình sinh dữ liệu, sử dụng các ưu điểm của các phương pháp trên đồng thời thêm các tính chất để phù hợp phù hợp với yêu cầu của bài toán. Dữ liệu được sinh ra thỏa mãn các điều kiện được xác định trong đặc tả ca sử dụng, giúp người dùng có thể thẩm định và xác minh ca sử dụng ngay trong quá trình thiết kế ca sử dụng. Đồng thời, việc biểu diễn dữ liệu được sinh ra dưới dạng biểu đồ đối tượng trên công cụ giúp trực quan hóa, làm dữ liệu dễ sử dụng cho các ca kiểm thử cho ca sử dụng.

Phương pháp sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng được đề cập trong khóa luận đã đạt được một số kết quả nhất định, thể hiện qua bài toán thử nghiệm cho thấy tính khả thi của việc xây dựng ca kiểm thử từ đặc tả ca sử dụng. Hướng phát triển tiếp theo của khóa luận là kết hợp với kỹ thuật đặc tả kịch bản kiểm thử chấp nhận để có thể tạo ra phương pháp sinh tự động kịch bản kiểm thử chấp nhận từ đặc tả mô hình ca sử dụng. Ngoài ra, tự động hóa hoàn toàn quá trình tạo các ràng buộc khung, thêm các chức năng thẩm định các ràng buộc OCL, khắc phục các hạn chế sẵn có là các mục tiêu tiếp theo của khóa luận.

Qua một số bài toán thử nghiệm, mặc dù còn nhiều hạn chế tuy nhiên kỹ thuật sinh tự động dữ liệu kiểm thử từ đặc tả ca sử dụng đã cho thấy tiềm năng phát triển rất lớn. Dù đã cố gắng hoàn thành đề tài tốt nhất có thể, khóa luận vẫn không thể tránh khỏi nhiều thiếu sót, em rất mong nhận được ý kiến đóng góp của mọi người để đề tài có thể hoàn thiện và vận dụng tốt vào bài toán thực tế.

DANH MỤC TÀI LIỆU THAM KHẢO

Tài liệu Tiếng Việt

Tài liệu Tiếng Anh

- [1] Sendall, Shane & Kozaczynski, Wojtek. (2003). Model Transformation: The Heart and Soul of Model-Driven Software Development. Software.
- [2] The Object Constraint Language (Second Edition) Author: J. Warmer, A. Kleppe Publisher: Addison-Wesley Professional
- [3] Desai, Nisha & Gogolla, Martin. (2019). Developing Comprehensive Postconditions Through a Model Transformation Chain.. The Journal of Object Technology.
- [4] Przigoda, Nils & Niemann, Philipp & Filho, Jonas & Wille, Robert & Drechsler, Rolf. (2017). Frame Conditions in the Automatic Validation and Verification of UML/OCL Models normalsize A Symbolic Formulation of modifies only Statements. Computer Languages, Systems & Structures.
- [5] Hilken, Frank & Niemann, Philipp & Gogolla, Martin & Wille, Robert. (2014). Filmstripping and Unrolling: A Comparison of Verification Approaches for UML and OCL Behavioral Models
- [6] Desai, Nisha & Gogolla, Martin. (2019). Developing Comprehensive Postconditions Through a Model Transformation Chain
- [7] Gogolla, Martin and Frank Hilken. (2016). Model Validation and Verification Options in a Contemporary UML and OCL Analysis Tool.
- [8] OMG Unified Modeling Language. <https://www.omg.org/spec/UML/2.5.1/PDF>
- [9] Marco Brambilla, Jordi Cabot, Manuel Winmmer. (2012). Model-Driven Software Engineering in Practice, Morgan & Claypool.
- [10] Craig Larman. (2004). “Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition”, Addison Wesley Professional.
- [11] Gogolla, M., Hamann, L., Hilken, F., Kuhlmann, M., & France, R.B. (2014). From Application Models to Filmstrip Models: An Approach to Automatic Validation of Model Dynamics. Modellierung.

- [12] Gogolla, Martin & Büttner, Fabian & Richters, Mark. (2007). USE: A UML-based specification environment for validating UML and OCL.
- [13] Hilken, Frank & Hamann, Lars. (2020). History of the USE Tool 20 Years of UML/OCL Modeling Made in Germany..